

北京交通大学

硕士专业学位论文

交通互操作语言语义理解方法研究

Research on semantic understanding methods for traffic
interoperability language

作者:

导师:

北京交通大学

2024 年 4 月

学校代码：10004

密级：公开

北京交通大学

硕士专业学位论文

交通互操作语言语义理解方法研究

Research on semantic understanding methods for traffic
interoperability language

作者姓名：

学 号：

导师姓名：

职 称：

专业学位类别：

学位级别：

北京交通大学

2024 年 4 月

摘要

自主式交通系统是以自感知、自适应、自学习、自组织为特征的高度自治的新一代交通系统，其包含了多种交通方式，并涉及到多种交通主体。而保障不同领域，不同类型交通主体之间能相互合作、协同工作的互操作技术正是构建自主式交通系统的基础。目前由于各种交通方式以及各类交通主体内部技术标准和通信协议的不统一，使得不同交通主体之间往往存在信息交互壁垒，难以满足自主式交通系统内部群体多交通主体综合协同认知需求。为了促使各个交通主体能够相互理解和协作，实现交通系统互操作，需要制定一套通用的交通互操作语言规范，以确保交通系统内部信息交互的准确性。目前关于交通互操作语言规范的设计已经趋于完善，本文的研究重点就是在此基础上为交通主体研究可以实现对交通互操作语言进行语义理解的方法。本文主要研究内容如下：

（1）本文为交通互操作语言设计了语义解析模型，其能将交通互操作语言转换为交通主体可以理解执行的语义表示。通过设计相应的词法分析器，语法分析器和语义分析器，逐步解析交通互操作语句中的语义信息，并设计策略执行模块，通过构造执行函数库，将所设计的语义解析模型，同后续语义推理模型所连接起来。该模型能够准确地理解交通互操作语句的意图和需求，并根据所提取出的语义信息调用相应的执行函数，是实现语义理解的基础。

（2）本文为交通互操作语言设计了语义推理模型，其允许交通主体基于预设的交通系统推理规则，在完成交通互操作语言语义解析的基础上进一步对交通知识进行推理和分析。基于层次状态机理论，按照交通方式和交通主体的不同，对自主式交通系统推理规则库进行了设计与分层。并通过对条件排列顺序的优化和网络结构的改进，提出了一种改进 RETE 算法。通过对比实验，本文所设计改进 RETE 算法在多规则匹配中性能较传统 RETE 算法有显著提升。

（3）本文为交通互操作语言语义理解模型设计了一系列仿真实验。基于 Carla 仿真软件以及 PYQT5 框架开发了交通互操作语言语义理解交互界面，并在多种道路交通场景下测试了所控制车辆对交通互操作语言中不同类型原子语句的响应表现，验证了所设计交通互操作语言语义解析模型和推理模型在道路交通环境中的适用性和有效性。

图 34 幅，表 13 个，参考文献 50 篇。

关键词：自主式交通系统，语义理解，规则推理，RETE 算法，Carla

ABSTRACT

Autonomous transportation system is a highly autonomous new generation of transportation system characterized by self perception, adaptation, self-learning, and self-organization. It includes multiple transportation modes and involves multiple transportation entities. The interoperability technology that ensures cooperation and collaborative work among different types of transportation entities in different fields is the foundation for building an autonomous transportation system. At present, due to the inconsistency of various transportation modes and internal technical standards and communication protocols among various transportation entities, there are often information exchange barriers between different transportation entities, making it difficult to meet the comprehensive collaborative cognitive needs of multiple transportation entities within autonomous transportation systems. In order to promote mutual understanding and cooperation among various transportation entities and achieve interoperability of transportation systems, it is necessary to develop a universal set of transportation interoperability language standards to ensure the accuracy of information exchange within the transportation system. At present, the design of traffic interoperability language specifications has become more perfect, and the focus of this article is to study methods for traffic entities to achieve semantic understanding of traffic interoperability language based on this foundation. The main research content of this article is as follows:

(1) This article designs a semantic parsing model for traffic interoperability languages, which can transform traffic interoperability languages into semantic representations that traffic agents can understand and execute. By designing corresponding lexical analyzers, syntax analyzers, and semantic analyzers, the semantic information in traffic interoperability statements is gradually parsed, and a strategy execution module is designed. By constructing an execution function library, the designed semantic parsing model is connected to subsequent semantic inference models. This model can accurately understand the intention and requirements of traffic interoperability statements, and call corresponding execution functions based on the extracted semantic information, which is the basis for achieving semantic understanding.

(2) This article designs a semantic inference model for traffic interoperability language, which allows traffic agents to further infer and analyze traffic knowledge

based on preset traffic system inference rules, on the basis of completing semantic analysis of traffic interoperability language. Based on the hierarchical state machine theory, the inference rule library of autonomous transportation systems was designed and layered according to the different modes of transportation and transportation entities. And by optimizing the order of conditional arrangement and improving the network structure, an improved RETE algorithm was proposed. Through comparative experiments, the improved RETE algorithm designed in this article has significantly improved performance in multi rule matching compared to traditional RETE algorithm.

(3) This article designs a series of simulation experiments for the semantic understanding model of traffic interoperability language. A traffic interoperability language semantic understanding interaction interface was developed based on Carla simulation software and PYQT5 framework. The execution of different types of atomic statements in the traffic interoperability language by the controlled vehicles was tested in various road traffic scenarios, verifying the applicability and effectiveness of the designed traffic interoperability language semantic parsing model and inference model in road traffic environments.

There are 34 figures, 13 tables and 50 references in this paper.

KEYWORDS: Autonomous traffic system; Semantic understanding; Rule reasoning; RETE algorithm; Carla

目录

摘要.....	III
ABSTRACT.....	IV
目录.....	VI
1 绪论.....	1
1.1 课题来源.....	1
1.2 研究背景与意义.....	1
1.3 国内外研究现状总结.....	2
1.3.1 交通系统互操作技术.....	2
1.3.2 语言编译技术.....	3
1.3.3 知识推理技术.....	4
1.4 研究现状总评.....	5
1.5 研目内容与结构组织.....	6
1.5.1 研究内容.....	6
1.5.2 论文组织框架.....	7
1.5.3 技术路线.....	8
2 交通互操作语言介绍.....	9
2.1 交通互操作语言字母表.....	9
2.1.1 逻辑符号.....	9
2.1.2 非逻辑符号.....	11
2.2 交通互操作语言语法规范.....	12
2.2.1 语法结构.....	12
2.2.2 句法结构.....	14
2.3 交通互操作语言知识推理.....	16
2.3.1 逻辑推理规则.....	16
2.3.2 逻辑知识推理示例.....	17
2.4 本章小结.....	19
3 交通互操作语言语义解析模型.....	21
3.1 交通互操作语言语义解析模型设计.....	21
3.1.1 交通互操作语言解析工具选择.....	21

3.1.2	交通互操作语言语义解析模型架构	22
3.2	词法分析模块	23
3.2.1	词法分析器构建	23
3.2.2	词法分析器测试	27
3.3	语法分析模块	28
3.3.1	语法分析器构建	28
3.3.2	语法分析器测试	32
3.4	语义分析模块	34
3.4.1	语义分析模块设计	34
3.4.2	语义分析模块测试	37
3.5	策略执行模块	38
3.6	本章小节	40
4	交通互操作语言语义推理模型	41
4.1	交通互操作语言语义推理模型设计	41
4.2	自主式交通系统推理规则研究	42
4.2.1	交通知识划分	42
4.2.2	推理规则设计	43
4.3	交通互操作语言逻辑推理算法研究	45
4.3.1	RETE 算法.....	45
4.3.2	RETE 算法改进策略设计.....	47
4.3.3	RETE 算法改进策略实现.....	50
4.3.4	改进 RETE 算法对比测试.....	52
4.4	本章小节	54
5	仿真实验验证	55
5.1	实验环境搭建	55
5.1.1	仿真平台选择	55
5.1.1	仿真环境的搭建	56
5.1.2	QT 界面开发设计	57
5.2	仿真实验设计	58
5.2.1	实验场景选择与设置	58
5.2.2	测试方案设计	59
5.3	仿真测试与分析	60

5.3.1	谓词语句.....	60
5.3.2	函数语句.....	61
5.3.3	动作语句.....	62
5.4	本章小结.....	64
6	结论与展望.....	65
6.1	研究结论.....	65
6.2	研究展望.....	65
	参考文献.....	67

1 绪论

1.1 课题来源

“十四五”国家重点研发计划“交通载运装备与 智能交通技术”重点专项，《自主式交通系统互操作技术（共性关键技术）》课题，多交通主体语义化表达与协同认知技术。

课题编号：2022YFB4300402

1.2 研究背景与意义

交通运输是国民经济中具有基础性，先导性，战略性的产业，是重要的服务性行业和现代化经济体系的重要组成部分^[1]。通过道路，水运，轨道等多种交通方式，交通运输系统实现了人员、货物和信息的快速、安全的流动，推动了贸易、产业和城市化的发展。同时交通运输系统的高效运作不仅提高了生产力和效率，还为社会各个领域的交流和合作提供了基础，对于推动经济增长、提升人民生活水平以及促进国际间的交流与合作具有不可替代的重要作用。在“十三五”期间，我国交通运输体系快速发展，建成了世界第一的高铁运营里程，世界第一的高速公路通车里程等瞩目成绩^[2]，而如今进入“十四五”时期，交通运输行业将继续坚持以习近平新时代中国特色社会主义思想为指导，落实新发展理念，紧扣高质量发展，构建新发展格局，加快建设人民满意、世界前列的交通强国，为全面建设社会主义现代化国家当好先行。

与此同时，新一代科学技术的迅速发展为整个交通系统的信息化和智能化带来了全新的机遇，传统的智能交通系统(Intelligent Transportation System,ITS)建设也迎来了新的机遇与变革。如何在诸多新兴技术助力下，建设能进行自组织运行与自主化服务的自主式交通系统(Autonomous transportation system,ATS)^[3]，成为了交通领域探索的新方向。

自主式交通系统的目标是尽可能减少交通活动中人为干扰的不确定性，通过其自组织运行和自主化服务的特性，将整个交通系统的不可控风险控制在最低，以期高效迅速的完成运输需求，从根本而言其是传统 ITS 架构在新兴技术加持下所完成的升级架构。整个自主式交通系统的运行逻辑是“自主感知，自主学习，自主决策，自主响应”^[4]，并以此为基础朝着实现“交通需求主动响应，载运工具自动运行，基础设施主动管控，外部环境主动适用”的发展目标迈进。区别于

传统交通系统里不同交通方式各自为政的管控方式，自主式交通系统将道路，铁路，水运等多种交通方式联通成一个整体，对这些交通场景里的交通主体进行统一管理和调度，以期通过对不同交通主体的协同管控促使整个交通系统达到资源配置最优，运行效率最高的效果。

目前不同交通系统内部以及其之间互操作性差是影响整个交通系统智能协同运行综合性能的关键问题，这一问题主要体现在不同或相同交通方式的不同交通主体(车辆，列车，船舶，路侧设施)交互困难之中。由于在发展的过程中，不同交通主体所面对的应用场景和管理体制不同，导致其通常会采用独立于其他交通主体的技术标准和通信协议，这就造成了整个交通系统内部各子系统在技术标准，构成要素，运作机制等方面存在差异，实际上使得这些子系统处于分离和孤立的状态。并且由于缺乏统一的交互标准，各个交通系统之间也无法通过交互实现信息共享，结构统一，这就造成了信息孤岛、系统集成难的情况^[5]。因此需要设计一门适用于交通系统域内和跨域交互的交通互操作语言，以期促进更广泛的信息共享与系统协同，引领交通系统向更智能、更协同的自主式交通系统发展。项目课题以此为背景开展研究，本文是在课题的框架内，研究适用于交通互操作语言的语义理解技术，其他关键技术由课题组其他成员负责完成。

1.3 国内外研究现状总结

1.3.1 交通系统互操作技术

自感知、自适应、自学习、自组织是自主式交通系统的特征，在构建这样的系统时，需要实现不同交通主体之间的有效合作与协同，互操作技术因此成为这一体系的构建基础。在传统意义上，互操作性是指不同类型的计算机系统、软件应用程序或网络能够无缝地、有效地交换和利用信息的能力^[6]，即“数据在不同平台或编程语言之间进行交换和共享的能力”。随着技术的进步和新兴概念的出现，互操作性的定义也在不断扩展，新概念下的互操作性不仅包括了技术层面的兼容与交互，还增加了对数据、语义、政策、法规和组织等更广泛层面的考虑。这意味着，新概念下的互操作性不仅关注技术之间如何互联互通，还关心数据如何被理解和利用、系统如何在不同的政策和法规框架内有效协作，以及不同组织之间如何通过标准化的方式共享和使用信息。

传统的交通系统互操作性研究往往仅局限于某一个交通方式内部，在铁路交通中，刘程民^[7]在高速列车网络系统研究中，定义了多节点间的互操作性，并探究了部件属性及其拓扑连接如何影响不同部件间的互操作性，基于引力模型，他

建立了一个用于实现高速列车部件间互操作性的模型。在水运交通中, Riccardo Costanzi 等人^[8]为提升无人海上船舶(UMVs)之间的互操作性,设计了一种新型的通信协议和数据共享框架,该框架能够有效处理和优化复杂海洋环境下的声学通信问题,包括通信延迟和多径干扰等。在道路交通中 Do-Hyun Kim 等人^[9]针对车辆遥感系统中不同信息提供商生成的数据格式不兼容,导致的内容整合与交换难度大问题,设计了一套遥感内容网关(TCG)系统。该系统通过支持基于 Web 服务的搜索功能以及提供一个共享的道路网络模型,实现了不同遥感内容之间的互操作性。

在自主式交通系统的背景下,互操作技术不仅需要确保不同交通系统在技术上能够互联互通,还需要确保它们能够理解和处理共享的数据,并遵循相应的安全和隐私政策。目前国内外研究人员已经针对自主式交通系统互操作性开展了大量的工作。刘光昫^[10]围绕自主式交通系统(ATS)架构的可靠性评估问题进行了深入研究,将 ATS 架构的可靠性评估与系统间的互操作性联系起来,并通过构建一个考虑物理、功能和逻辑架构的评估模型,对整个系统的可靠性进行了评估。Huang Xiangzhi 等人^[11]为了更清晰地展示自主式交通系统的运行方式,按照不同的功能以基于文本分析的方式将自主式交通系统功能划分为不同的功能域,尽量减少对主观经验的依赖,为自主式交通系统互操作性层级划分提供了参考。

1.3.2 语言编译技术

对于交通互操作语言而言,实现对其的语义理解实际就是研究构建能够对其进行解析的编译模型,它通过将交通互操作语言编写的语句描述转换为所有交通主体都能够理解和执行的格式,确保了不同交通主体之间能够准确无误地交换信息。这个过程不仅涉及到对词法语法的检查和转换,同时更深层次涉及到对语句语义信息的理解,包括其在相应交通场景下的含义和目标。通过这样的转换,能为自主式交通系统构建高效协作的交通管控系统提供支撑,使得各种交通主体即便拥有着独立的通信协议和交互标准,也能够依托交通互操作语言实现共享与协作,以提升系统互操作性,提高运行效率和安全性。目前对于语言编译模型的构建主要是基于如下几种经典框架。

其中比较经典的语言编译模型构建框架为三段式结构^[12],由前端,优化器和后端构成,其中前端负责处理源语言的语法和语义分析,这一阶段将源代码转换成一个中间表示,通常是抽象语法树或其他形式的中间表示。优化器在中间表示上执行各种优化,以提高程序的执行效率。后端将优化后的中间表示转换成目标语言,这可以是机器代码、字节码或其他形式的低级代码。采用三段式框架的最

为经典例子为 GCC。GCC 是由 Richard Stallman^[13]在 1987 年发起的 GNU 项目的一部分，目的是创建一套完全自由的软件开发工具集。GCC 是一个开源的编译器套件，支持多种编程语言，如 C、C++、和 Java 等，它遵循传统的三阶段编译过程，包括前端、优化器和后端，旨在提供跨平台的编译支持。

随着编程语言的多样化和目标平台的扩展，传统的编译模型设计框架开始显得不够灵活，难以快速适应新的技术需求，因此模块化编译模型框架应运而生。模块化框架将编译过程分解成独立的模块或组件，每个模块负责编译过程中的一个特定任务。这种设计使得编译模型可以更容易地扩展和维护，也便于复用某些模块。LLVM (Low Level Virtual Machine)^[14]是一个著名的模块化编译器框架，提供了丰富的库和工具链，用于开发编译器前端、优化器和后端。LLVM 项目，最初作为 Chris Lattner 在 2000 年的博士论文项目发布，后来成为了一个广泛支持的开源编译器基础设施项目。LLVM 旨在提供一套模块化的编译器技术和工具链，支持静态和动态编译的各种语言。

1.3.3 知识推理技术

知识推理是自主式交通系统不可或缺的技术功能，通过运用人工智能领域的先进技术，如逻辑推理、机器学习和大数据分析等^[15]，提升整个自主式交通系统的智能化程度。基于此，自主式交通系统能模拟人类的逻辑思维过程，处理并推理复杂的交通知识，通过对现有知识的分析来推导未知信息或潜在关系，以解决所面临的管控难题。这一技术不仅能极大地提高了自主式交通系统的安全性和效率，也为实现交通系统的全面智能化迈出了关键一步。根据不同的标准可以将知识推理技术划分为多种类型，其中每种类型都有其特定的应用场景和优势，以下是几种常见的知识推理技术类型，以及关于其近几年在交通领域的应用研究。

符号推理是一种传统的知识推理方法，也称为基于规则的推理，其通过预定义的逻辑规则来推导新的知识。虽然符号推理在处理结构化知识方面表现出色，但它在处理大规模知识库和自然语言文本时仍存在不足。Gilpin 等人^[16]探讨了在交通领域构建神经符号系统的意义，设计了一个包含安全性、感知和诊断/复杂推理三个类别在内的交通任务预测分类体系，并在自动驾驶和交通监控任务中进行了测试，旨在通过神经符号推理连接不同模态，提高推理过程的可解释性。Kolla 等人^[17]使用从大型语料库和知识图谱中提取到的常识知识，研究构建了两个文本型多项选择问答集，用于评估在交通领域进行因果推理的能力以及对交通领域相关知识的掌握程度。

图推理是一种基于图结构数据进行推理的知识推理方法，其利用图的节点和边来表示和处理关系密集的知识集合，从而实现复杂的推理任务。张召霞^[18]为解决无人驾驶车辆行为决策的知识获取与表示、知识推理和存储等关键问题，开展了基于多级知识超图的行为决策知识表示、知识一致性检验、多知识融合推理，以及面向无人驾驶车辆行为决策知识库管理系统构建方法等方面的研究，实现了计算机理解用户查询条件语义后的自动进行推理。

概率推理则是利用概率模型来处理不确定性信息，为知识的不确定性提供了一种量化的处理方法。James R. Ward 等人^[19]提出了一种基于车辆间通信系统的推理方法，用于预测和防止车辆碰撞，该方法构建了一个概率推理框架对所有内容进行整合，并能够考虑到车辆状态的不确定性和车辆间通信的可能丢失状况，从而提供了一种新的视角来估计车辆碰撞风险。Carlos R.C.Souze 等人^[20]设计了一个基于马尔可夫逻辑网络的交通场景概率逻辑推理系统，为驾驶辅助系统提供车辆在运行过程中定位和行为信息的高级解释。

1.4 研究现状总评

综上所述，国内外诸多研究人员已经对上述各项技术在交通领域的应用开展了大量研究，为本文的研究提供了宝贵的参考和支撑。但是就对自主式交通系统互操作性的研究而言，各学者目前开展的工作往往仍是局限在单一交通领域内的研究，但理论上以自主式交通系统为背景的研究应当考虑到多种交通方式和各种交通主体。因此，仅仅将研究限定在单一交通方式内，单独研究水运，道路或轨道领域的互操作性，是无法解决自主式交通系统所面临的互操作性差的问题。此时对于跨域和域内交互行为就需要一个中间语言作为不同交通主体进行交互的语义信息载体，交通互操作语言因此需要被设计出来。

同时结合上述研究分析，目前语言编译模型构建所采用的主流构建框架包括基于模块化的编译框架和传统的三段式框架，前者提供了极高的灵活性和可维护性，适合于需要支持多种语言或多个目标平台的复杂项目。而后者则在保证编译效率和优化效率的同时相应支持了跨平台编译，但在灵活性和对特定需求的定制性方面略逊一筹。

而在知识推理方面，不同的知识推理方法依据其自身独特的优势都在交通场景中得到了广泛的应用，符号推理的优势在于其精确性和逻辑严密。图推理的优势在于能够直观地处理交通状态的动态变化。概率推理的优势则是能利用统计和概率模型预测未来的交通状况，尤其擅长处理不确定性和随机性高的交通预测和安全评估问题。

考虑到交通互操作语言语义理解方法的可解释性和模型效率，本文设计的语义解析方法将会采用基于模块化的编译模型架构，一方面其不仅能依靠模块化特性提高开发和维护的效率，另一方面也由于其结构的明确性，进而增强了语义解析过程的可解释性。同时，本文设计的语义推理模型将会采用基于规则推理的语义推理机制，其能为交通主体所进行的每一步推理提供了明确的逻辑依据，不仅有助于提升语义推理模型的可信度，还能够在复杂的交通环境中提供高效且准确的语义推理结果，提升了自主式交通系统的安全性和可靠性。

1.5 研目内容与结构组织

1.5.1 研究内容

针对承载着自主式交通系统内跨域和域内语义动作交互需求和状态信息传递需求相关语义信息的交通互操作语言，分析并实现对其语义理解方法的研究，本文的具体研究内容如下：

(1) 交通互操作语言语义解析模型

针对所处理的交通互操作语言，本文依托于模块化的编译模型架构，完成对交通互操作语言词法分析，语法分析，语义理解，语句执行模块的设计，构建了一个高效准确的语义解析模型，从而实现对交通互操作语句的初步解析。这一模型的核心目的是通过对交通互操作语言语义信息的准确解析，保障不同交通系统不同交通主体之间的通信效率和信息交换的准确性，为实现高度自动化和智能化的自主式交通系统奠定基础。

(2) 交通互操作语言语义推理模型

针对语义解析模型得到的各种语义信息，本文设计了一个基于规则推理的语义推理模型，规定了预设推理规则的规范描述格式，并基于层次状态机理论对整个自主式交通系统的推理规则库进行了分层，同时，为推理模型设计了一种改进 Rete 算法。这一模型的核心目的是通过对交通互操作语言语义信息的进一步推理，从中挖掘出交通系统更深层次的行为模式和潜在的变化趋势，赋予自主式交通系统更为智能的决策依据。

(3) 基于 Carla 仿真的交通场景仿真应用

基于 Carla 仿真软件和 Pyqt5 框架开发了交通互操作语言语义理解交互界面，实现了包括语句输入，视角切换在内的多种功能，并在此基础上通过输入不同类型的交通互操作语言原子语句对所设计的语义解析模型和语义推理模型进行了仿

真测试验证，根据控制车辆的响应表现，评估整个语义理解模型对交通互操作语言的解析和推理能力。

1.5.2 论文组织框架

本论文共分为五个章节，各个章节的具体内容安排如下所示：

第一章：引言

本章介绍了本论文选题的相关背景和意义，并对国内外相关研究现状进行了总结分析，明确了论文整体的研究内容和研究目标，同时对整个论文的组织框架和技术路线进行了设计。

第二章：交通互操作语言介绍

本章系统地介绍了交通互操作语言的字母表、语法规则和推理规则。并通过对一个道路推理示例的详细描述，展示了交通互操作语言是如何作为自主式交通系统互操作语义信息载体来表达知识和传递信息，表现了交通互操作语言强大的表达能力和推理能力。

第三章：交通互操作语义解析模型

本章详细的介绍了交通互操作语言语义解析模型的构建流程，包括解析模型框架设计和模块化结构实现。展示了如何通过该模型逐步检查和提取交通互操作语言中的语义信息，并如何将得到的语义信息传递给语义推理模型，表现了解析模型对交通互操作语言的准确解析。

第四章：交通互操作语义推理模型

本章详细的介绍了交通互操作语言语义推理模型的构建流程，包括推理模型框架设计、推理规则构建、推理算法改进等。展示了该模型如何利用解析出的语义信息进行高效的知识推理，以支持语义动作的执行，表现了推理模型对交通互操作语言语义的深度挖掘。

第五章：仿真测试与验证

本章详细的介绍了交通互操作语言语义理解模型的仿真测试过程，包括仿真环境搭建、测试场景设计以及测试方案选择等。通过一系列仿真实验，展示了不同交通场景下控制车辆对不同交通互操作语句的响应表现，证明了语义理解模型在处理交通互操作语言方面的准确性和可靠性。

第六章：总结与展望

本章归纳总结了本文的主要研究成果和主要结论，并且针对论文研究中的不足之处，提出了下一步的研究改进方向。

1.5.3 技术路线

本文技术路线图如下图 1-1 所示。

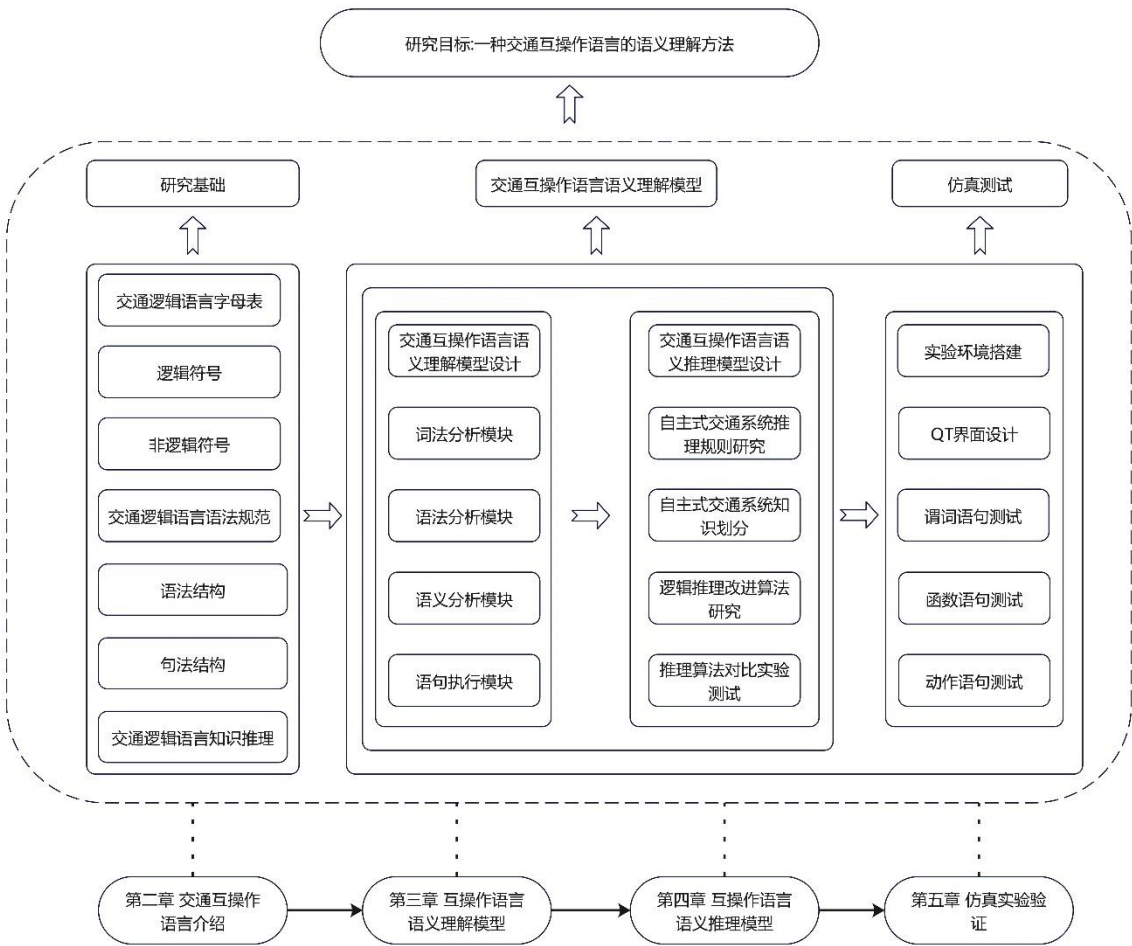


图 1-1 论文整体框架

Figure 1-1 The overall framework of the paper

2 交通互操作语言介绍

针对各交通方式交通主体之间语义表达不一致的难题，同时为了实现交通状态信息知识表达模型的构建，并完成对交通状态知识的语义化描述和推理，需要为整个自主式交通系统制定相应的交通互操作语言规范，本章将会详细介绍交通互操作语言的字母表以及语法规则，同时以道路交通为背景举例说明交通互操作语言的知识推理过程。

2.1 交通互操作语言字母表

交通互操作语言又称交通逻辑语言，其是一种基于一阶逻辑设计，用来描述交通系统中各种信息和关系的形式化语言，而字母表则表示了组成其语句的各种基本符号类型，具体可以划为逻辑符号(logical symbols)和非逻辑符号(nonlogical symbols)两种类型：

2.1.1 逻辑符号

(1) 个体

在一阶逻辑的框架下，个体被理解为所研究领域内独立存在的实体，无论是具体的还是抽象的^[21]。在自主式交通系统的背景下，个体特指系统里不同的交通主体。而个体词进一步可以细分为个体常量和个体变量，它们用于表示交通领域中的各种元素，这些元素在句子中充当主语或宾语的角色，是交通互操作语言中用来表达具体实体或泛化交通概念的基本符号。

个体常量用于指代具体的、特定的对象，例如交通系统中的道路名称或车辆标识，这些对象往往是固定的，不变的，设定其由不同的大写字母开头的字符区分不同类型的交通主体（如 Trafficlight, Car, ...）表示，同时通过下划线后接数字对同一类型的不同交通主体进行划分（如 Car_1, Car_2, ...）表示。

个体变量代表更为抽象或泛指的概念，比如某一类车辆的速度或加速度，这些对象往往是实时变化的，设定其由小写字母开头的字符来表示（如 speed, brake, ...），以便于表达交通主体下细分的泛化的概念并进行逻辑推理，同时需要后接括号表示个体变量所属的交通主体（如 speed(Car_1), brake(Car_1), ...）。

(2) 逻辑连接词

逻辑连接词用于连接不同的交通逻辑表达式，以构建更加复杂的交通逻辑关

系。在交通互操作语言里参照一阶逻辑共设置有“与 $\wedge$ ”、“或 $\vee$ ”、“非 $\neg$ ”、“蕴含 $\Rightarrow$ ”、“等价 $\Leftrightarrow$ ”五种逻辑连接词，以此能够表达更具体和复杂的交通知识和交通规则，如下例所示，同时表 2-1 展示了上述逻辑连接词所对应的真值表。

例如，“如果车辆 1 到达车道 1 时车道 1 信号灯为绿灯，那么车辆 1 可以通行”可以表示为“ $\text{IsOnLane}(\text{drivelane}(\text{Car_1}), \text{Lane_1}) \wedge \text{IsGreen}(\text{color}(\text{Trafficlight_1})) \Rightarrow \text{Accelerate}(\text{Car_1})$ ”。

表 2-1 逻辑连接词真值表
Table 2-1 A truth table of join words

A	B	$\neg A$	$\neg B$	$\neg A \vee B$	$\neg B \vee A$	$A \Leftrightarrow B$
T	T	F	F	T	T	T
T	F	F	T	F	T	F
F	T	T	F	T	F	F
F	F	T	T	T	T	T

(3) 量词

量词可以实现对多种交通对象属性的同时表达，而不需要根据名称逐个列举交通对象，量词是表示常量或变量之间数量关系的词，其包括全称量词和存在量词^[22]，全称量词表示域内所有的对象， $\forall$ 通常与蕴含符号 $\Rightarrow$ 一同出现。

例如， $\forall \text{Car}(\text{IsRedLight}(\text{color}(\text{Car})) \Rightarrow \text{Stop}(\text{Car}))$ 表示“这里，Car 代表任何一个车辆， $\text{IsRedLight}(\text{color}(\text{Car}))$ 判断车辆主体是否面临红灯， $\text{Stop}(\text{Car})$ 表示此类车辆主体执行停车动作，全称量词 $\forall \text{Car}$ 表示规则适用于所有面临红灯的车辆。存在量词表示域内存在相关对象， $\exists$ 通常与合取符号 $\wedge$ 一同出现。 $\exists \text{Car}(\text{IsOverSpeed}(\text{speed}(\text{Car})) \wedge \text{IsLightOff}(\text{light}(\text{Car})))$ 这里，Car 代表车辆主体， $\text{IsOveSpeed}(\text{speed}(\text{Car}))$ 判断是否有车辆超速， $\text{IsLightOff}(\text{light}(\text{Car}))$ 判断是否有车辆车灯未开，若上述表达式为真，则表达了至少有一辆车同时满足这两个条件。

同时可以使用多个量词对更复杂的语句进行表示，即嵌套量词。最简单的情形是量词种类相同的情形，例如，“在每一个交叉路口车辆需要减速”可以写成 $\forall \text{Car} \forall \text{Intersection}(\text{IsNearIntersection}(\text{Distance}(\text{Car}, \text{Intersection})) \Rightarrow \text{Decelerate}(\text{Car}))$ ，些特殊情况下允许使用混用量词，但应注意量词的顺序变化会对语义产生巨大的影响，而且当多个量词与相同的变量名合用时会引起混淆，因此规定所提及的个体常量属于提及它的最内层量词，随后便不再受任何其他量词约束，并且始终在嵌套量词中使用不同的个体常量名。

(4) 括号

在交通互操作语言里，括号用于明确整个互操作语句的结构，特别是在涉及

量词和逻辑运算符时。例如，全称量词 ($\forall$) 常与蕴含符号 ($\Rightarrow$) 一起使用来表达适用于交通领域里某一域内的所有对象。括号在这种表达中非常关键，其帮助界定量词的作用范围和逻辑运算符的优先级，确保表达式的意义准确无误。例如，互操作语句 " $\forall \text{Car}(\text{IsRedLight}(\text{color}(\text{Car})) \Rightarrow \text{Stop}(\text{Car}))$ " 中的括号清楚地表明，蕴含关系适用于每一个由量词 $\forall$ 定义的个体常元 Car，其中以车辆面临红灯为真为前提，车辆停车动作为推理。这种使用避免了逻辑上的混淆，使得整个表达式的逻辑结构清晰可见。

2.1.2 非逻辑符号

(1) 谓词符号

谓词符号用来表示交通领域中的关系或属性，其根据一定数量交通参数进行判断，同时输出布尔值（真或假），用来判断交通参数所构成的组合是否满足某种指定的条件或关系。在交通互操作语言中，谓词同个体词一样也有常项和变项之分，表示具体条件或关系的谓词称为谓词常项，表示抽象的、泛指的性质或关系的谓词称为谓词变项，二者均以大写字母 Is 开头后接表示其判断内容的标识符，同时后接括号并列出其所需要检查的个体变量，并需要具体注明个体变量所属的个体常量。

例如， $\text{IsAccelerate}(\text{speed}(\text{Car}_1), \text{limitedspeed}(\text{Lane}_1))$ 中谓词 IsAccelerate 需要检索常量 Car₁ 和 Lane₁ 的 speed 和 limitedspeed 变量，来进行推理判断，并返回 True 或 False。

(2) 函数符号

函数符号代表了一个数学函数，它接受一定数量的交通参数输入，并且映射到另一个交通参数的输出。这个输出通常涉及到具体的交通对象。函数符号本质上可以理解为通过已有的交通属性去构建出新的交通属性。在交通互操作语言中函数符号需要自行定义关键词进行设置，同时其所涉及到的相关交通主体需要在其后续括号内字母中列出，因为对于函数符号而言，其每个函数都有特定的处理逻辑，因此只需列出主体即可。一般地，有 n 个参数的函数被称作是 n 元 (n-ary) 的，n 是函数的元数 (arity) ^[23]，在逻辑推理语言中函数一般在谓词内部使用，用于计算某些无法之间获取的交通知识，为谓词推理提供相应参数。

例如， $\text{Distance}(\text{Car}_1, \text{Trafficlight}_1)$ 中函数 Distance 为计算不同个体常量之间的具体，通过输入具体的个体常量如车辆 1 和交通信号灯 1，返回车辆 1 距离交通信号灯 1 的距离。

(3) 动作符号

动作符号代表了交通主体所要具体执行的动作，其可以看作是特殊的函数符号，但不同的是除了注明动作所执行的交通主体以外还需要传入通过一系列预期交通参数，以确保其对交通指令的准确执行。在交通互操作语言里，其也是自行定义关键词进行设计，但是其不返回值而且直接按照所传入的预期交通参数对控制交通主体进行控制。通过动作符号能够进一步实现交通主体对语义的执行，因此其往往作为推理结论出现。

例如 `Accelerate(Car_1, target_speed, target_acceleration)` 中动作 `Accelerate` 为加速动作，要求车辆 `Car_1` 执行加速动作，同时需要传入相应的预期参数，包括，目标速度和目标加速度。

2.2 交通互操作语言语法规则

2.2.1 语法规则

(1) 项

项 (term) 是指代对象的逻辑表达，不同的个体常元与个体变元自身就是一个项。由于使用不同的符号来命名每个对象可能会比较复杂，因此，引入了复合项的概念以简化表示，复合项由一个非逻辑符号（如函数名）引导，后接一对括号，括号中包含了作为该符号参数的一系列项。这样的定义使得复合项成为表示整个非逻辑符号调用的方式，并直接关联到该非逻辑符号所对应的运算。

例如，复合项 `AverageSpeed(Car_1, ..., Car_n)`，通过函数符号 `AverageSpeed` 表达了计算某个路段所有车辆的平均速度，这里的参数项 `Car_1` 到 `Car_n` 表示所需要纳入平均速度计算的车辆主体。整个复合项表示的是将函数 `AverageSpeed` 应用到这些车辆的速度变量上，从而得到的平均速度值。

(2) 公式

1) 原子公式

原子公式 (atomic formula) 是形式为 $R(t_1, \dots, t_n)$ 的公式，其中 R 为非逻辑符号，表示是一个 n 元交通关系，而 $t_1, \dots, t_n$ 都是项。原子公式可表示一个简单命题，其中非逻辑符号 R 表示该命题所要表达的关系，而其后面的各个项表示这一关系所涉及的具体交通变量。同时可将原子公式按照其引导词的不同进行细分，分为谓词原子公式，函数原子公式和动作原子公式，此外的任何交通互操作语句都是由这三类原子语句通过逻辑连接词（如全称量词、存在量词、逻辑与、逻辑或、蕴含等）所组合形成的，这样能对更为复杂的状况进行更为全面的逻辑表示，适于分析、推理和规划交通系统的行为。

例如, $\text{IsAccelerate}(\text{speed}(\text{Car_a}), \text{limitedspeed}(\text{Lane_a}))$ 即为一个谓词原子公式, 其中 IsAccelerate 表示进行谓词推理, 括号内的两个项分别代表执行推理时需要检索车辆主体的行驶速度和车道主体的限速信息。

在逻辑表示中, 原子公式扮演着描述事实或状态基本单元的角色, 用于表示关于特定对象或情况的简单陈述, 提供了逻辑表示中的基本构建块, 可以用于描述交通系统中的各种属性、条件和关系。并且在逻辑推理中, 原子公式也可以作为基础事实, 用于推导更复杂的逻辑结论^[25], 亦或是充当推理的结论。

2) 合式公式

合式公式也称为一阶逻辑公式, 简称公式。合式公式是由原子公式通过逻辑联结词或者量词所构建的更为复杂的语句。在命题逻辑中, 把一些固定符号看作是原语 (primitive), 其他符号都由这些符号定义。而在一阶逻辑中, 把通常将逻辑联结词和量词视为原语, 例如 $\neg$, $\wedge$, $\exists$ 等^[26], 参照一阶逻辑, 交通互操作语言的所有合式公式都通过重复利用下列规则从原子公式建造而来, 每条规则对应相应原语。

- ①如果 ϕ 是公式, 那么 $\neg\phi$ 是公式。
- ②如果 ϕ 和 ψ 都是公式, 那么 $\phi \wedge \psi$ 和 $\phi \vee \psi$ 是公式。
- ③如果 ϕ 是公式, 那么对于任意变量 x , $\exists x(\phi)$ 和 $\forall x(\phi)$ 是公式。
- ④如果 ϕ 和 ψ 都是公式, 那么 $\phi \Rightarrow \psi$ 和 $\phi \Leftrightarrow \psi$ 也是公式。

同时在公式 $\forall x(A)$, $\exists x(A)$ 中, 跟在量词后面的个体变元 x 称为指导变元, 公式 A 称为这两个量词的辖域。而个体变元出现的类别可以分为约束出现和自由出现。约束出现的变元称为约束变元, 自由出现的变元称为自由变元。不含自由变元的公式称为封闭公式 (closed formula) 或者语句 (sentence), 含自由变元的公式称为开放公式 (open formula) 或者句型 (sentential form)^[27]。

例如, $\forall \text{Car}(\text{IsSafeSpeed}(\text{speed}(\text{Car})) \Rightarrow \text{safedriving}(\text{Car}))$, 这个合式公式表示了一个逻辑推理规则: 如果所有车辆的速度都不超过安全限速即 IsSafeSpeed 为真, 则所有车辆的安全驾驶属性 safedriving 都为真。这个公式可以在交通系统中用于推断是否需要发出安全警告或者采取车辆控制措施。

合式公式在逻辑表示中用于描述交通系统中更为复杂的规则、条件和关系。这些公式可以用于推理、规划和决策, 帮助自主式交通系统在面对极为复杂的交通场景时能保持其强大的推理和决策能力, 并且通过逻辑表示, 可以对交通系统进行建模、分析和优化, 从而提高交通系统的效率、安全性和可靠性。

3) 子公式

子公式 (subformula) 是指构成复合公式一部分的任何公式。在形式逻辑中, 如果公式 ϕ 包含另一公式 θ , 则 θ 被称为 ϕ 的子公式。这个定义可以应用于交通

互操作语言的上下文中，以帮助分析和处理具体的交通情况。对于交通互操作语言，可以归纳性地定义子公式如下。

- ①如果 φ 是原子公式，那么 θ 是 φ 的子公式当且仅当 $\theta=\varphi$ 。
- ②如果 φ 具有形式 $\neg\psi$ ，那么 θ 是 φ 的子公式当且仅当 $\theta=\varphi$ 或 θ 是 ψ 的子公式。
- ③如果 φ 具有形式 $\psi_1 \vee \psi_2$ 或 $\psi_1 \wedge \psi_2$ ，那么 θ 是 φ 的子公式当且仅当 $\theta=\varphi$ 或 θ 是 ψ_1 或 ψ_2 的子公式。
- ④如果 φ 具有形式 $\psi_1 \Rightarrow \psi_2$ 或 $\psi_1 \Leftrightarrow \psi_2$ ，那么 θ 是 φ 的子公式当且仅当 $\theta=\varphi$ 或 θ 是 ψ_1 或 ψ_2 的子公式。
- ⑤如果 φ 具有形式 $\exists x(\psi)$ 或 $\forall x(\psi)$ ，那么 θ 是 φ 的子公式当且仅当 $\theta=\varphi$ 或 θ 是 ψ 的子公式。

4) 语句

交通互操作语言的语句 (sentence) 定义是一个没有自由变量的公式。其不包含任何自由变量，这意味着所有变量都通过量词进行了绑定，这使得语句的真值不依赖于外部的变量赋值，而是可以单独确定。例如： $\exists x \forall y (P(x,y) \vee Q(x,y))$ 即为语句。

与自由变量相反，使用约束变量 (bound variable) 来指代没有量词的变量。对于任意一阶公式 φ ， $\text{bnd}(\varphi)$ 表示出现在 φ 中的约束变量组成的集合。可以归纳地定义约束变量如下：

- ①如果 φ 是原子公式，那么 $\text{bnd}(\varphi)=\emptyset$ ，
- ②如果 $\varphi=\neg\psi$ ，那么 $\text{bnd}(\varphi)=\text{bnd}(\psi)$ ，
- ③如果 $\varphi=\psi \wedge \theta$ 或 $\varphi=\psi \vee \theta$ ，那么 $\text{bnd}(\varphi)=\text{bnd}(\psi) \cup \text{bnd}(\theta)$ 。
- ④如果 $\varphi=\exists x(\psi)$ 或 $\varphi=\forall x(\psi)$ ，那么 $\text{bnd}(\varphi)=\text{bnd}(\psi) \cup \{x\}$ 。

所有出现在 φ 中的变量都在 $\text{free}(\varphi)$ 或 $\text{bnd}(\varphi)$ 中。实际上，这两个集合不必须是不相交的。一个变量在一个公式中可能既自由的出现又约束的出现。

在交通互操作语言中，语句起着描述命题或陈述的作用，它们是逻辑推理和分析的基本单位^[28]。语句通常用于表达某种断言、规则、条件或事实，帮助理解和描述各种情况和关系。

2.2.2 句法结构

在逻辑表示中，句法结构通常用来表达一个命题或陈述的意义^[29]。将指代对象的项以及指代关系的非逻辑符号结合起来可以构成陈述事实的原子语句，也可以用逻辑联结词构建更为复杂的语句。原子语句及复合语句的句法分析是交通互操作语言处理中的基础性工作，通过句法分析，可以为交通领域中交通场景的语

义分析、规则抽取等打下坚实的基础，规定句法结构就是确定交通互操作语言所能支持的语句结构如主谓宾、定状补等，并能在此基础上进一步分析各成分语义之间的关系，设定的句法结构的标注如下表 2-2 所示：

表 2-2 句法结构分析标注
Table 2-2 Syntactic structure analysis annotation

关系类型	描述	例子	逻辑语言表示
主谓关系	Subject-verb	车辆 a-加速	Accelerate (Car_1,target_speed)
动宾关系	Verb-object	查询-速度	InspectSpeed (Car_1)
间宾关系	Indirect-object	发送-信号-车辆 b	SendSignal(Car_1, Car_2)
前置宾语	Fronting-object	车道数-多于两条	lanecount(Road_1) > 2
定中关系	Attribute	超速-车辆	IsOverSpeed(speed(Car_1))
状中关系	Adverbial	极其-危险	IsExDangerous(state(Lane_1))
动补关系	Complement	加入-车队	Join(Car_1,Quene_1)
并列关系	Coordinate	同时-到达	Arrive(Car_1) $\wedge$ Arrive(Car_2)
介宾关系	Preposition	停在-停车场	IsInParkingLot(location(Car_1), location(ParkingLot_1))

以下是一个简单的交通互操作语言逻辑句法结构的示例：

假设需要表达一个关于道路交通系统的命题：“如果车辆在路口等待绿灯，同时信号灯变绿，则车辆可以通过路口。”为了用交通互操作语言描述这个命题，可以利用谓词、量词和逻辑连接词进行表述。以下是对应的交通互操作语句：

- (1) 使用个体常量 Car 来表示车辆，而 Trafficlight 表示交通信号灯。
- (2) 定义谓词 IsWaiting(state(Car))来判断车辆是否处于等待状态。
- (3) 定义谓词 IsGreen(color(Trafficlight))来判断交通信号灯是否处于绿灯状态。
- (4) 定义动作 Canpass(Car,target_lane)来为车辆设定目标车道并允许其通过路口。

结合主谓关系和动宾关系，可以得到以下逻辑规则表达式：

$\forall \text{Car} \forall \text{Trafficlight} (\text{IsWaiting}(\text{state}(\text{Car})) \wedge \text{IsGreen}(\text{color}(\text{Trafficlight})) \Rightarrow \text{Canpass}(\text{Car}, \text{target_lane}))$

这个逻辑表达式的含义是：对于所有的车辆 Car 和所有的交通信号灯 Trafficlight，如果某个车辆 Car 正在停车等待且相应的信号灯 Trafficlight 显示为绿色，则该车辆 Car 可以通过路口并进入目标车道。基于上述语法规则的表达式

不仅逻辑严密，还符合一般的逻辑表达式书写规范，能够清楚地传达命题的意图。

2.3 交通互操作语言知识推理

2.3.1 逻辑推理规则

交通互操作语言将采用一阶逻辑的推理规则进行推理，具体的推理规则如下所示：

(1) 合取引入 (Conjunction Introduction):

如果 P 为真且 Q 为真，则 P 与 Q 的合取为真。

逻辑表示: $P \wedge Q$

例子: 如果“ A 在交叉路口左转”(P) 为真且“ B 也在交叉路口左转”(Q) 为真，则“ A 和 B 都在交叉路口左转”为真。

(2) 合取消去 (Conjunction Elimination):

如果 P 与 Q 的合取为真，则 P 为真且 Q 为真。

逻辑表示: $P \wedge Q \Rightarrow P, P \wedge Q \Rightarrow Q$

例子: 如果“ A 和 B 都在交叉路口直行”为真，则“ A 在交叉路口直行”(P) 为真且“ B 在交叉路口直行”(Q) 为真。

(3) 假言推理 (Implication Introduction):

如果 P 成立导致 Q 成立，那么如果 P 成立，则 Q 成立。

逻辑表示: $P \Rightarrow Q$

例子: 如果“信号灯是绿灯”(P) 为真则“允许通行”(Q) 为真。

(4) 蕴含消去 (Implication Elimination):

如果 P 成立导致 Q 成立，且 P 为真，则可以得出 Q 为真。

逻辑表示: $P \wedge (P \Rightarrow Q) \Rightarrow Q$

例子: 如果“信号灯是绿灯”(P) 能推出“允许通行”(Q)，那么若“信号灯是绿灯”为真，则“允许通行”为真。

(5) 拒取引入 (Negation Introduction):

如果 P 导致矛盾，则非 P 为真。

逻辑表示: $P \Rightarrow \text{contradiction} \Rightarrow \neg P$

例子: 如果当前不允许通行，这使得其与“信号灯是绿灯”(P) 矛盾，则“信号灯不是绿灯”($\neg P$) 为真。

(6) 拒取消去 (Negation Elimination):

如果非 P 为真，则 P 为假。

逻辑表示： $\neg P \Rightarrow P = \text{False}$

例子：如果“信号灯不是绿灯”（ $\neg P$ ）为真，则“信号灯为绿灯”（ P ）为假。
可以利用这些规则来对交通场景下的各种情况和结论进行推理。

2.3.2 逻辑知识推理示例

通过交通互操作语言实现对交通知识进行结构化表达后，可以基于预设的交通规则和各交通主体状态信息，通过知识推理来推导相关结论。本文将以交叉口红灯预警规则为例，演示整个逻辑推理过程：

（1）场景描述

车辆Car以特定的加速度 $\text{acceleration}(\text{Car})$ 和速度 $\text{speed}(\text{Car})$ 向交通信号灯Trafficlight行驶。车载智能终端实时接收有关车辆Car和交通信号灯Trafficlight各项数据，包括位置 location ，距离 $\text{Distance}(\text{Car}, \text{Trafficlight})$ ，交通信号灯在 t 时刻的相位状态 $\text{lightstate}(\text{Trafficlight})$ 以及剩余时间 $\text{timeleft}(\text{Trafficlight})$ 等。同时监测道路状况 $\text{condition}(\text{Lane})$ ，交通拥堵情况 $\text{congestion}(\text{Trafficlight})$ ，行人横穿状况 $\text{crossing}(\text{President})$ ，以及天气状况 $\text{weathercondition}(\text{Trafficlight})$ 。

（2）推理过程

车辆主体可以通过函数 $\text{Arrivetime}(\text{Distance}, \text{speed})$ 计算预计到达交叉口时间，若 $\text{Isnearintersection}(\text{Arrivetime}, \text{signalperiod}(\text{Trafficlight}))$ 为真，即到达时间小于一个信号周期，则激活交叉口红灯预警推理，包括两种可能的情况：

1) 当交通信号灯Trafficlight在此时相位状态为绿灯或黄灯时，车辆Car以当前车速 $\text{speed}(\text{Car})$ 匀速行驶至交通信号灯Trafficlight的时间 Arrivetime 大于信号灯绿灯剩余时间+黄灯时间；

2) 当交通信号灯Trafficlight在此时相位状态为红灯时，车辆Car以当前车速 $\text{speed}(\text{Car})$ 匀速行驶至交通信号灯Trafficlight的时间函数 Arrivetime 小于红灯剩余时间。

基于情景中车辆与信号灯所获取到的各项知识，当所有条件被满足时，触发闯红灯预警，信号灯主体会向车辆主体发出预警信息，提醒存在闯红灯风险，车辆以当前速度无法正常通过交叉口：

$$\begin{aligned} & \text{IsgoodRoadcondition}(\text{condition}(\text{Lane})) \wedge \neg \text{Iscongestion}(\text{congestion}(\text{Trafficlight})) \\ & \wedge \neg \text{Iscross}(\text{crossing}(\text{President})) \wedge \text{Issunny}(\text{weathercondition}(\text{Trafficlight})) \wedge \neg \text{IsPass} \\ & (\text{Arrivetime}(\text{Distance}(\text{Trafficlight}, \text{Car}), \text{speed}(\text{Car})), \text{timeleft}(\text{Trafficlight})) \Rightarrow \text{AlertTrigger} \\ & (\text{Car}, \text{Trafficlight}) \end{aligned}$$

在触发了红灯预警规则后，进一步会触发交叉口决策规则，会根据当前交通

信号灯状态，对所需要进行的驾驶决策进行推理：

1) 如果交通信号灯当前显示为绿灯或黄灯，通过逻辑推理能推测车辆Car是否有足够的时间和距离在不超速的情况下顺利通过信号控制路口。

①加速通过

如果通过推理，判断在信号灯时长，车辆速度以及道路限速允许的条件下，可以通过加速通过交叉口，那么会返回相应的动作符号，其中包括车辆通过交叉口的预期速度以及加速度：

$$\text{IsRedAlert}(\text{AlertTrigger}(\text{Car}, \text{Trafficlight}), \text{Color}(\text{Trafficlight})) \wedge \text{IsPass}(\text{speed}(\text{Car}), \text{limitspeed}(\text{Lane})) \Rightarrow \text{Accelerate}(\text{Car}, \text{target_speed}, \text{target_acceleration})$$

②减速停车

如果通过推理，判断车辆无法在信号变更前安全通过，那么会要求车辆减速到停车线前停车，那么会返回相应的动作符号，其中包括车辆停到停车道的预期减速度：

$$\neg \text{IsRedAlert}(\text{AlertTrigger}(\text{Car}, \text{Trafficlight}), \text{Color}(\text{Trafficlight})) \wedge \neg \text{IsPass}(\text{speed}(\text{Car}), \text{limitSpeed}(\text{Lane})) \Rightarrow \text{Dccelerate}(\text{Car}, \text{target_speed}, \text{target_deceleration})$$

2) 如果交通信号灯当前显示为红灯，通过逻辑推理能推测车辆Car是否能在不停车的前提下顺利通过信号控制路口。

①减速停车

如果通过推理，判断交通信号灯红灯剩余时间较长，无法不停车通过信号控制路口，那么会返回相应的动作符号，其中包括车辆停到停车道的预期减速度。

$$\text{IsRedAlert}(\text{AlertTrigger}(\text{Car}, \text{Trafficlight}), \text{Color}(\text{Trafficlight})) \wedge \text{IsRedPass}(\text{timeleft}(\text{Trafficlight}), \text{speed}(\text{Car}), \text{Distance}(\text{Car}, \text{Trafficlight})) \Rightarrow \text{Dccelerate}(\text{Car}, \text{trget_speed}, \text{target_deceleration})$$

②不停车通过

如果通过推理，判断交通信号灯红灯的剩余时间较短，车辆可以保持在一定速度在不完全停车的情况下顺利通过交叉口，那么会返回相应的动作符号，其中包括车辆顺利通过交叉口的预期速度以及加速度或减速度。

$$\text{IsRedAlert}(\text{AlertTrigger}(\text{Car}, \text{Trafficlight}), \text{Color}(\text{Trafficlight})) \wedge \text{IsRedPass}(\text{timeleft}(\text{Trafficlight}), \text{speed}(\text{Car}), \text{Distance}(\text{Car}, \text{Trafficlight})) \Rightarrow \text{Accelerate}(\text{Car}, \text{trget_speed}, \text{target_aceleration}) \vee \text{Dccelerate}(\text{Car}, \text{trget_speed}, \text{target_deceleration})$$

通过推理知识使得车辆主体能够根据实时数据动态调整整个车辆控制系统的目标策略，从而减少交通事故的风险，确保车辆顺利通过信号交叉口，提高道路使用效率和车辆的安全性。

2.4 本章小结

本章深入探讨了交通互操作语言的语言学规则，通过引入一套全面且严格的语义概念体系，来解决不同交通方式不同交通主体之间互操作难的问题。同时详细介绍了交通互操作语言的字母表，语法规范和推理规则，并通过交叉口红灯预警规则推理展示了交通互操作语言强大的表达能力及其在复杂交通场景下的推理全过程。

3 交通互操作语言语义解析模型

作为一门新设计的语言，交通互操作语言语义理解的核心在于其语义解析模型的实现，否则在交通主体接受到其他交通主体传输过来的互操作语言时，是无法对其进行语义信息提取和分析的，更不具备将其执行的能力。因此本章基于 Python 开发了交通互操作语言语义解析模型，并基于模块化的架构设计将整个语言的解析过程划分为四个功能模块：词法分析模块，语法分析模块，语义分析模块和策略执行模块，最终构建起交通互操作语言语义解析模型。

3.1 交通互操作语言语义解析模型设计

3.1.1 交通互操作语言解析工具选择

语义解析技术是交通互操作语言设计中关键一环，通过选择合适的解析工具对交通互操作语言进行解析，可以发挥出交通互操作语言强大的知识表达能力，从而满足整个交通系统的互操作需求。因此本文首先调研了目前在逻辑语言分析中常使用的解析工具，并进行了归纳总结如下表 3-1 所列。

表 3-1 解析工具调研
Table 3-1 Analysis tool survey

工具名称	类型	特点	应用场景
ANTLR4	Parser Generator	支持 LL(*) 文法，全能且灵活	编译器、解释器、DSL 实现等
Yacc/Bison	LALR(1) 文法解析器生成器	底层语法分析器，用于编译器构建	编译器开发、底层语法分析器
PEG.js	PEG 的解析器生成器	直观语法规则定义，适用于 Web 开发	领域特定语言（DSL）解释器，前端开发
PLY	词法分析器和语法分析器生成器	灵活，Python 中集成，适用于小型项目	Python 项目的语法分析器、编译器开发
Jison	JavaScript 版本的 Bison	生成 JavaScript 解析器，适用于前端开发	与浏览器端交互的语言解析器，前端开发

根据对上述解析工具的调研分析，本文将选择 PLY(Python Lex-Yacc)作为实现交通互操作语言语义解析的工具，主要基于其几个显著的优势。首先，PLY 与

Python 的紧密集成为基于 Python 的开发提供了无缝的开发体验，可以轻松地将 PLY 与现有的 Python 代码库和各种第三方库结合使用，极大地增强了其适用性和灵活性。此外，PLY 的高度可定制性和强大的错误检测能力使得它非常适合于处理复杂的语义理解任务，能够有效地对解析过程中产生的问题进行诊断和处理^[30]。最重要的是，本文第五章实例分析中所使用到的 Carla 软件提供的是 Python 接口，通过使用 PLY 进行语义解析模型的设计能使得所设计的模型可以直接同本文第五章实例分析中所使用到的 Carla 软件进行对接，更便于对所设计的交通互操作语言语义解析模型进行仿真验证。

综上所述，本文将选择 PLY 作为交通互操作语言语义解析模型的构造工具。

3.1.2 交通互操作语言语义解析模型架构

交通互操作语言语义解析模型基于 PLY 开发，遵从模块化编译模型架构，将整个模型划分为四个模块，其中前三个模块依次为词法分析模块，语法分析模块，和语义分析模块，这三个模块会逐步提取交通互操作语言的语义信息，使得互操作语言能有效发挥其作为语义载体的作用，此外还构造了策略执行模块，通过设计执行函数库，联通互操作语言语义解析模型和语义推理模型，使得交通主体在对所接受到的互操作语言处理的同时，能实现对所获取语义信息进行进一步推理，下图 3-1 展示了整个语义解析模型的总体架构。

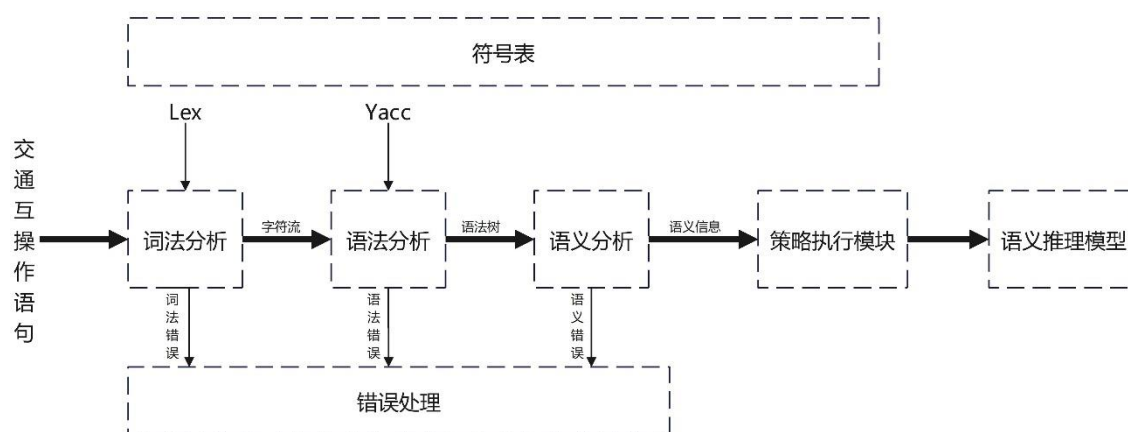


图 3-1 语义解析模型模型总体架构

Figure 3-1 The overall architecture of the Semantic Parsing Model

3.2 词法分析模块

3.2.1 词法分析器构建

词法分析模块是交通互操作语言语义解析模型的首要模块，其负责处理输入的交通互操作语句，依据一组预定义的词法规则进行，逐字符地从左到右扫描，并从中识别和提取出有意义的词元。词法分析的结果是将输入的文本字符串转换成一系列词元，这些词元随后会作为记号流被传递给语法分析器进行下一步的语法解析。大致的步骤如下图 3-2 所示：

- (1) 词法分析器按从左到右的顺序接受输入语句中的字符，并根据当前状态和读入的字符，决定下一步的状态转移。
- (2) 根据交通互操作语言词法规则设计相对应的的正则表达式进行词元匹配，当匹配成功时，会为相应字符生成相应的词元。
- (3) 随着词元匹配的结束，所生成的词元将汇总成一个序列，这个序列便为语法分析器提供了所需的单词记号流，用以进一步的语言语义解析。

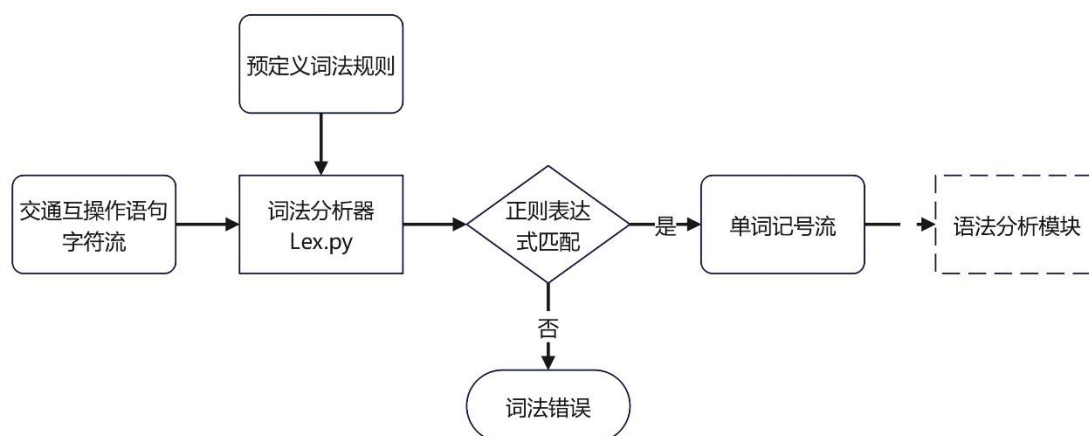


图 3-2 词法分析过程

Figure 3-2 Lexical analysis process

根据上述步骤安排，为了便于后续构建，首先需要对所涉及到的两个重要理论进行介绍，分别是正则表达式和有穷自动机理论。

(1) 正则表达式

正则表达式（Regular Expression）是一种强大的文本处理工具，用于匹配、查找、替换文本中的特定模式^[31]，它通过定义一个搜索模式，实现对字符串进行复杂的搜索、编辑和处理。下表 3-2 列出了正则表达式所包含的模式内容。

表 3-2 正则表达式模式内容
Table 3-2 Regular expression patterns

模式	内容
字面值字符	例如字母、数字等，可以直接匹配它们自身
特殊字符	例如星号*、问号?等，它们具有特殊的含义和功能
字符类	用方括号[]包围的字符集合，匹配方括号内任意一个字符
元字符	例如 \d、\w 等，用于匹配特定类型的字符
量词	例如 {n}、{n, m} 等，用于指定匹配的次數或范围
边界符号	如 ^、\$、\b 等，用于匹配字符串开头、结尾或单词边界

不同的程序语言往往有具有其特定的词法规则，同时按照这些词法规则产生的单词符号都是一些特殊的字符串，因此，可以对相应的词法规则进行形式化描述，即描述词法规则所对应的单词集合，此时正规式即是词法规则一种形式化描述方法，对应的单词集合则称为正规集^[32]，下面是对正规式与正规集的递归定义：对于给定的字母表 Σ ：

- 1) ϵ 和 $\emptyset$ 都是 Σ 上的正规式，则它们所表示的正规集是 $\{\epsilon\}$ 和 $\{\emptyset\}$
- 2) 任何 $a \in \Sigma$ ， a 是 Σ 上的正规式，则它所表示的正规集是 $\{a\}$
- 3) 若 e_1 和 e_2 都是 Σ 上的正规式，则它们所表示的正规集为 $L(e_1)$ 、 $L(e_2)$ ，并且如下表 3-3 所列，也均是其正则表达式和所表示的正规集。

表 3-3 正规表达式及其对应的正规集合
Table 3-3 Regular expressions and their corresponding regular sets

正规式	正规集
(e_1)	$L(e_1)$
$e_1 e_2$	$L(e_1) \cup L(e_2)$
$e_1 \cdot e_2$	$L(e_1)L(e_2)$
e_1^*	$(L(e_1))^*$

综上所述，仅由上述三个步骤通过有限次使用定义的表达式才是 Σ 上的正规式，同时仅由这些正规式表示的字集才是正规集。

(2) 有穷自动机

有穷自动机 (Finite Automaton, FA)，也称为有限状态机，是一种抽象的计算模型，用于表示和控制执行流程^[33]。在计算理论、语言处理、硬件设计等领域中

有广泛的应用，有穷自动机能根据是否接受输入的字符来改变状态，进而决定接受或拒绝一个字符串，其定义可以用一个五元组的形式来表示，如公式 1 所示：

$$M = (Q, \Sigma, \delta, q_0, F) \quad (1)$$

式中 Q ——状态集合，包括所有可能的状态；

Σ ——输入字母表，是可接受的输入符号集合；

δ ——转移函数，定义从一个状态到另一个状态的转移规则，通常依赖于当前状态和输入符号；

q_0 ——初始状态，是自动机开始执行时的状态；

F ——接受状态集合，表示当自动机在这些状态之一中停止时，它接受输入的字符串。

对于有穷状态机的识别过程可以用相应的状态转移图进行描述，如图 3-3 所示，图中一共有 3 个结点，分别代表了 S_0 ， S_1 ， S_2 三种状态，起始状态为 S_0 ，终止状态为 S_2 用双圈标记，此时三个节点之间通过有向边进行连接，表示可以通过相应的状态转移函数进行状态的变化。

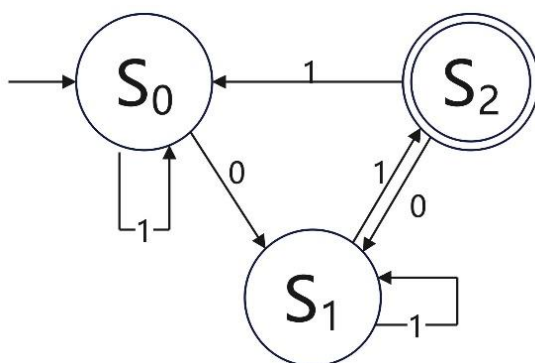


图 3-3 有限状态机状态转移图

Figure 3-3 Finite state machine state transition diagram

同时有限状态机根据状态迁移的确定性可以分为确定性有限状态机 (Deterministic Finite Automaton, DFA) 和非确定性有限状态机 (Nondeterministic Finite Automaton, NFA) [34]。在 DFA 中，每个状态对于输入符号表中的每个符号都有一个明确的、唯一的转移路径，这意味着给定当前状态和任何输入符号，DFA 都能确定下一个唯一的状态。而在 NFA 中，对于输入符号表中的某些符号，可能存在多条转移路径，或者在某些情况下，对于同一个符号，在同一状态下可以不进行任何输入即自动转移到另一个状态，图 3-3 实现的就是一个非确定性有限状态机。

结合上述理论，开始对交通互操作语言的词法规则进行定义，参照本文第二章对交通互操作语言字母表的介绍，可以将交通互操作语言词元类型分为如下几种：

(1) 个体常量：个体变量用于表示可变的实体，按照规范个体变量以小写字母开头后通过下划线接数字编号，如 Car₁。

(2) 个体变量：个体常量用于表示特定的、不可变的实体，按照规范个体变量由小写字母组成后接括号指定交通主体，如 speed(Car)

(3) 谓词符号：谓词符号用来判断后续括号内交通参数的关系，按照规范以字母 Is 开头，如 IsOverSpeed

(4) 函数/动作符号：函数/动作符号通常涉及到后续括号内交通参数的具体运算，按照约定以大写字母开头，自行定义关键词集合。

(5) 字符串：字符串表示文本，通常被双引号包围，字符串用于表示文本数据。

(6) 逻辑运算符：用于对原子公式进行运算，包括非($\neg$)、与($\wedge$)、或($\vee$)、蕴含($\Rightarrow$)、等价($\Leftrightarrow$)。

(7) 标点符号：逗号(,)用于分隔参数或条件，括号(())用于改变运算的优先级或分组，以及其他常用符号。

其次需要按照上述内容描述出交通互操作语言构词规则所对应的正则表达式，同时要设计相应的语义动作，使得词法分析器既可以实现对交通互操作语言进行词法分析，同时在识别到不同的词法单元时还执行相关语义动作。下表 3-4 展示了部分词元种类的正则表达式表示：

表 3-4 正则规则式匹配表

Table 3-4 Regular rule matching table

词元类型	正则表达式	词元类型	正则表达式
DOT	.	AND	$\wedge$
COMMA	,	OR	$\vee$
LPA	(	ALL	$\forall$
RPA	)	EXI	$\exists$
CON	$\backslash b[A-Z][a-zA-Z0-9_]*(?:[0-9]+)?\backslash b$	VAR	$\backslash b[a-z]+\backslash b$
PRE	$\backslash bIs[A-Z][a-zA-Z]*\backslash b$	ACT	Accelerate
STR	$\backslash ".*?"$	FUN	Distance

3.2.2 词法分析器测试

通过对交通互操作语言词法的解析，本文已经完成了对交通互操作语言词法解析器的构建，为验证所设计词法分析器的效用，本文将进一步对其解析交通互操作语言的效用进行分析，下图 3-4 展示交通互操作语言词法分析器的工作流程。

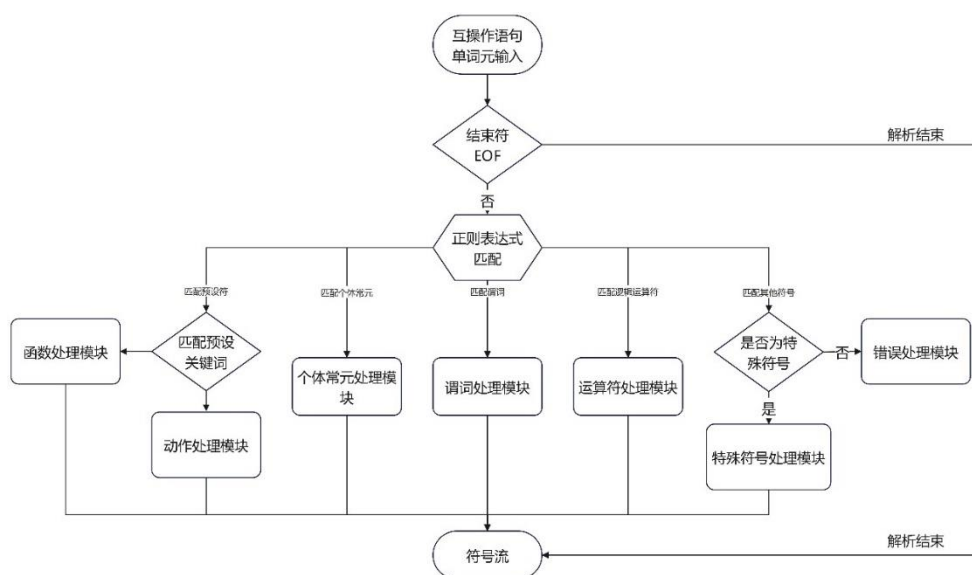


图 3-4 图元识别过程

Figure 3-4 Element recognition process

为了对设计好的词法分析器进行测试，本文将开启 LEX.py 库的词法调试模式，对交通互操作语言词法分析过程中的每个步骤进行测试。在词法调试模式中，LEX.py 将显示出它所匹配到的每条词法规则及相应的词素，从而能展示词法分析器在处理交通互操作语言时执行的具体行为，同时还能够检查是否有未设计到的字符或模式，以便进一步改进词法分析器的设计。

在调试模式下词法分析器会反复调用 `lex.token()` 函数，每次调用都会从输入字符流中按顺序选取一个有效 token，并且返回的每个 token 都是一个 `LexToken` 类型的对象，该对象包含四个主要属性：`t.type` 代表 token 的类型，`t.value` 是 token 的值，`t.lineno` 表示 token 所在的行号，而 `t.lexpos` 是 token 在输入文本中的偏移量，下图 3-5 展示了在调试模式下对某一具体交通互操作语句进行词法分析所输出的记号序列。

交通互操作语句：
∃Car(IsOverSpeed(speed(Car))∧IsLightOff(light(Car)))
词法解析：

Token	Type	Value	Line	Position
∃	EXI	∃	1	0
Car	CON	Car	1	1
(	LPA	(	1	4
IsOverSpee	PRE	IsOverSpee	1	5
(	LPA	(	1	16
speed	VAR	speed	1	17
(	LPA	(	1	22
Car	CON	Car	1	23
)	RPA	)	1	26
)	RPA	)	1	27
∧	AND	∧	1	28
IsLightOf	PRE	IsLightOf	1	29
(	LPA	(	1	39
light	VAR	light	1	40
(	LPA	(	1	45
Car	CON	Car	1	46
)	RPA	)	1	49
)	RPA	)	1	50
)	RPA	)	1	51

图 3-5 交通互操作语句词元解析
Figure 3-5 Morphological parsing of traffic interop statements

3.3 语法分析模块

3.3.1 语法分析器构建

语法分析模块是交通互操作语言语义解析模型中，位于词法分析之后的一个关键模块。其主要功能是将词法分析生成的词元标记流作为模块的输入，按照预设语法规则对其进行语法检验，并为下一步语义解析生成相应的抽象语法树 ATS。具体的步骤如下图 3-6 所示：

- （1）语法分析器逐个接收由词法分析模块输出的词元流，其中每个词元包括了具体的类型和相应的字面值；
- （2）根据交通互操作语言语法规则设计对应的 BNF 范式进行语法匹配,当匹配成功时会逐层构建起语句的层次结构；
- （3）当词元按照语法规则成功组合时，语法分析器会构建一个抽象语法树（AST），来表示交通互操作语言的语法结构，其中不同节点代表整个语言体系的不同构造层级。

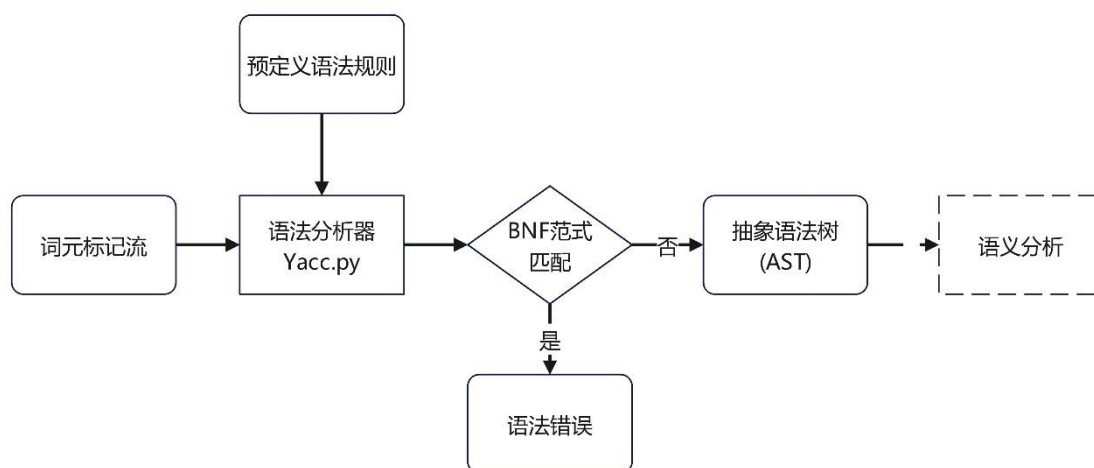


图 3-6 语法分析过程

Figure 3-6 Syntax analysis process

根据上述步骤安排，为了便于后续构建，首先需要对所涉及到的两个重要理论进行介绍，分别是上下文无关文法和自底向上的分析法。

（1）上下文无关文法

上下文无关文法（Context-Free Grammar, CFG）是计算机科学中用于形式语言描述的一种形式语法^[35]。它提供了一种方式来描述所有可能的字符串在某种特定语言中的语法结构。上下文无关文法在编程语言的设计、编译器的开发、以及自然语言处理等领域中有广泛的应用，其定义可以用一个四元组的形式来表示，如公式 2 所示。

$$G = (V, \Sigma, R, P) \quad (2)$$

式中 V --非空的终结符记号，是语法中的基本符号，不能被替换。

Σ --非空的非终结符记号，是语法的变量，可以通过应用产生式规则来替换。

R --文法的开始符号，是整个语言或表达式的起始；

P --有限个产生式子集合，定义了非终结符如何被终结符或其他非终结符的序列替换。

而巴克斯范式（BNF）则是一种用于描述上下文无关文法的形式化表示方法^[36]，这也是本文所采用对交通互操作语言语法规则进行描述的方法。

（2）自底向上的分析法

语法分析本质上进行的就是判断输入的词元标记流是否满足所预设的语法规则的工作，而要想判断一个句子是否某一语言，一般可以通过两种角度：句子的产生（推导）和句子的识别（归约）^[37]。产生句子的方式是从文法的开始符号开

始，逐步推导出该单词序列，那么则称为自顶向下的语法分析；而识别句子的方式则是逐步将构成程序的单词序列归约为文法的开始符号，就称为自底向上的语法分析。由于本文语法分析所使用的 Yacc 模块采用的分析法为前者，因此本文将重点介绍自底向上的分析法。

自底向上分析法采用移进-归约的框架实现^[38]，这种方法的核心思想是从叶子节点（即源代码中的基本元素如变量、常量、操作符等）开始构建树，逐步向上构建，直到达到根节点，这个根节点代表了整个源代码。其中所提及的“移进”是指将输入的下一个符号推入到一个栈中，而“归约”是指根据一定的语法规则将栈顶的几个符号合并成一个更高级的语法单位，基于这种思想可以构建出相应的语法分析器，其总体架构如图 3-7 所示。

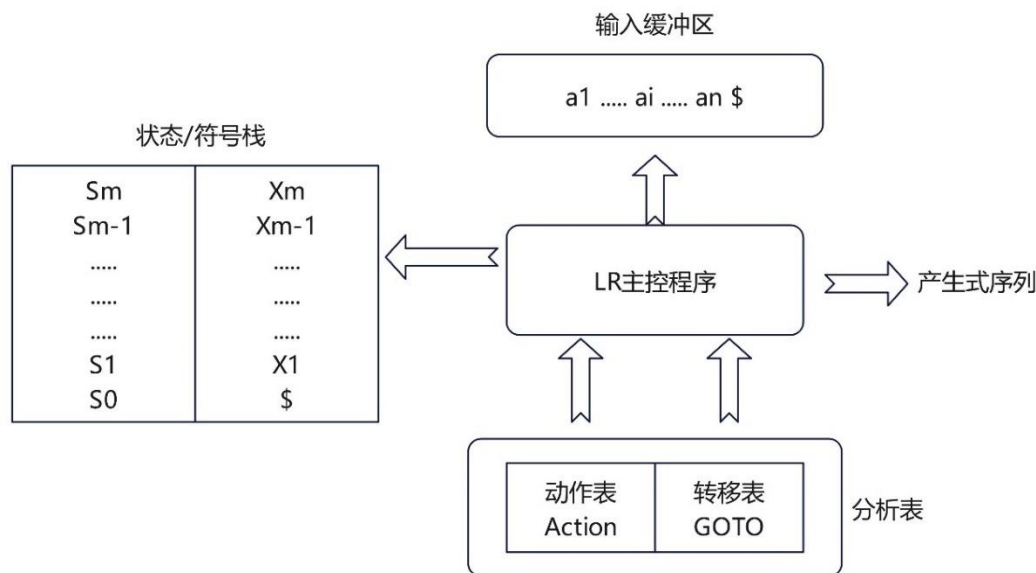


图 3-7 LR 分析器总体架构

Figure 3-7 The overall architecture of the LR analyzer

结合上述理论，开始对交通互操作语言的语法规则进行定义，参照本文第二章对交通互操作语言语法规则的介绍，可以将交通互操作语言语法规则分为如下几种：

- （1）公式：交通互操作语言由一系列子公式构成，公式是交通互操作语言的最顶层结构，表示了一个完整的交通互操作语句。
- （2）原子公式：原子公式是公式最基本的组成部分，其均由非逻辑符号引导，具体包括函数(FUN), 谓词(PRE), 动作(ACT)三种符号引导的原子公式。
- （3）逻辑公式：逻辑公式使用基本的逻辑运算符来组合交通互操作语言中的其他公式，支持多种逻辑运算，允许其构建复杂的逻辑关系。

(4) 量词公式：量词公式在交通互操作语言中引入了数理逻辑中的全称量词和存在量词，允许其能表达所有或存在某种情况的断言。

(5) 函数：函数是交通互操作语言中表达函数运算和动作指令的基本方式。其是预定义的字符类型，包括函数(FUN) 或动作(ACT)，用于执行具体操作。

(6) 参数列表：参数列表分为常量参数列表，和变量参数列表，用于定义原子公式中括号内的输入参数。它可以是单个标识符或通过逗号分隔的多个标识符，支持递归结构以匹配任意数量的参数。

(7) 变量结构：根据语法规则，个体变量的表示有固定表示方法，即变量后接括号并在括号内注明相应交通主体。

其次需要按照上述内容描述出交通互操作语言语法规则所对应的 BNF 范式，使得语法分析模块基于此可以实现对输入交通互操作语言标记字符流进行语法规则匹配，如下表 3-5 是部分语法规则的 BNF 范式表示：

表 3-5 BNF 范式表达
Table 3-5 BNF paradigm expression

非终止符	BNF 范式
<formula> 公式	<quantifier_formula> <logical_formula> <atom_formula>
<logical_formula> 逻辑公式	formula AND formula formula OR formula formula IMP formula formula EQU formula
<quantifier_formula> 量词公式	ALL CON LPA formula RPA EXI CON LPA formula RPA
<atom_formula> 原子公式	PRE LPA variable_list RPA function LPA constant_list RPA
<function> 函数	FUN ACT
<variable_list> 变量参数列表	variable variable_list COMMA variable
<variable> 变量结构	VAR LPA CON RPA
<constant_list> 常量参数列表	CON constant_list COMMA CON

在抽象语法树的构造方面，PLY 库本身并不直接提供专门用于构建语法树的工具或类，但是 PLY 提供了一套构建机制，允许程序能在解析过程中按照这套机制完成对语句抽象语法树的构建，并可以定义相关解析规则以及在这些规则匹配

时的执行动作。因此本文基于这套构建机制，手动为交通互操作语言设计了构建其抽象语法树的工具，具体流程如下所示：

（1）为交通互操作语言语法树中所涉及到的各种元素（如表达式、语句、运算等）定义 Python 类，这些类将会作为树的子节点。

（2）在每个解析规则的动作部分，根据匹配到的语法片段创建相应的节点实例，并设置节点间的关系（如父子关系）。

（3）通过递归或迭代执行解析规则，自底向上构建整个语法树。从最底层的叶节点开始，逐步通过解析规则合成更高层的节点，直到达到根节点。

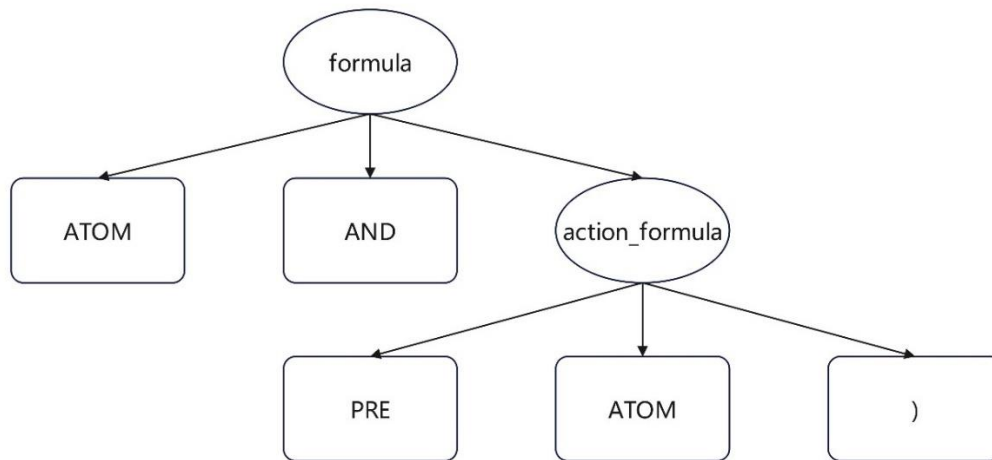


图 3-8 抽象语法树结构

Figure 3-8 Abstract syntax tree structure

3.3.2 语法分析器测试

通过对交通互操作语言语法的解析，本文已经完成了对交通互操作语言语法解析器和语法树构造工具的构建，为验证所设计语法分析器的效用，本文将进一步对其解析交通互操作语言的效用进行分析，下图 3-9 展示交通互操作语言语法分析器的工作流程。



图 3-9 语法分析器工作流程

Figure 3-9 Parser workflow

为了对设计好的语法分析器进行测试, 本文将开启 `Yacc.py` 库的语法调试模式, 对交通互操作语言语法分析过程中的每个步骤进行测试。通过语法调试模式, `Yacc.py` 将显示出分析器状态的详细转换过程, 从而能测试语法分析器在处理交通互操作语言时所执行的操作, 同时还能检测是否存在语法冲突或语法缺陷, 以便进一步改进语法分析器的设计, 在调试模式下, 语法分析器会逐个将标记词元移进栈内, 并且每压入一个词元就会检测栈内的词元是否满足预设的语法规范并实时将满足语法规则的符号合并成一个更高级的语法单位, 如下图展示了对 3-10 展示了在调试模式下某一具体交通互操作语句进行语法分析所执行的部分移入-规约过程。

```

交通互操作语句:
 $\exists \text{Car}(\text{IsOverSpeed}(\text{speed}(\text{Car})) \wedge \text{IsLightOff}(\text{light}(\text{Car})))$ 
语法解析:
Parsing method: LALR
state 1
  (0) S' -> formula .
  (4) logical_formula -> formula . AND formula
  (5) logical_formula -> formula . OR formula
  (6) logical_formula -> formula . IMP formula

  AND          shift and go to state 11
  OR           shift and go to state 12
  IMP          shift and go to state 13

state 2
  (1) formula -> quantifier_formula .

  AND          reduce using rule 1 (formula -> quantifier_formula .)
  OR           reduce using rule 1 (formula -> quantifier_formula .)
  IMP          reduce using rule 1 (formula -> quantifier_formula .)
  $end         reduce using rule 1 (formula -> quantifier_formula .)
  RPA          reduce using rule 1 (formula -> quantifier_formula .)

```

图 3-10 交通互操作语句语法解析

Figure 3-10 Syntax analysis of traffic interop statements

3.4 语义分析模块

3.4.1 语义分析模块设计

在确认交通互操作语言词法语法合规后，需要进一步对其语义信息进行检测，这时所进入的模块就是语义分析模块，其核心任务是依据语法分析阶段生成的抽象语法树（AST）对其进行静态语义检查，主要包括检查交通互操作语言的语义一致性，比如类型检查，个体声明检查以及逻辑检查等，并进行一些语义层面的转换或优化，以便为后续模块提供准确的语义信息，大致的步骤如下图 3-11 所示：

（1）依据交通互操作语言的语义定义，确定交通互操作语言中所有符号的解释集合，如谓词对应真假或个体常元对应的交通主体。确保语义分析的一致，为后续的类型检查和声明验证提供必要的前提条件；

（2）通过遍历从语法分析阶段获得的抽象语法树（AST），对每个节点进行类型检查，确保操作符两侧类型兼容，以及函数调用时传递的参数类型正确。同时，维护个体作用域规则，检查个体常量与个体变量是否在符号表中声明；

（3）分析交通互操作语言中可能的控制流结构，确保逻辑上的合理性和正确性，避免出现无限循环等问题并确保所有可能的执行路径最终都有返回值。并检查逻辑表达式的一致性和推理的合理性，以防止出现逻辑上的错误。

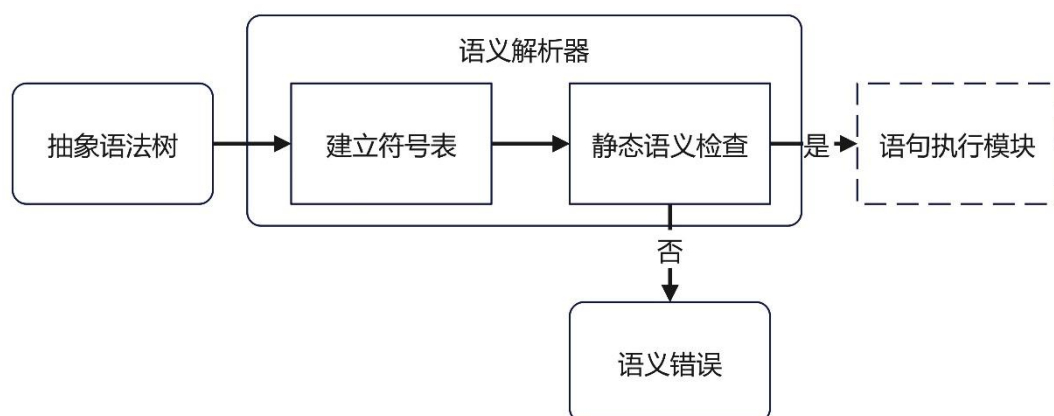


图 3-11 语义解析过程

Figure 3-11 Semantic parsing process

根据上述步骤安排，为了便于后续构建，首先需要对所涉及到的重要理论进行介绍，即是属性文法理论。

属性文法（Attribute Grammar）是一种形式化方法，用于定义语言的语法以及与语法结构相关联的语义信息^[39]。其是上下文无关文法（Context-Free Grammar, CFG）的拓展，使得语义操作能与相应的语法规则进行结合起来。属性文法其为每个文法符号（终结符或非终结符）设置了相关的属性，如类型，值以及相应的符号表类型，同时为文法的每一个产生式即 BNF 范式设计了相应的一套语义动作，以实现对子节点属性的计算和传递。

属性文法的属性可以分为两种类型，分别是综合属性和继承属性^[40]，前者是由一个节点的子节点的属性计算并综合得到的属性，其反映了从下至上的信息流动，常用于构建表达式的值、类型检查或者其他需要从底部向上聚合信息的场景。后者则是从一个节点的父节点和/或兄弟节点继承而来的属性，其反映了从上至下或者横向的信息流动，对于实现那些依赖于上下文信息的语义分析特别有用，比如变量的作用域分析、类型检查时需要的上下文信息等。这两种属性类型共同支持起了编译器内部复杂的语义规则表达，使得编译器能够执行深入的语义分析和代码生成任务，并且通过属性依赖实现属性与语义规则的绑定，其中属性依赖是指，若对应于每一个产生式 $A := a$ 都有一套与之相关联的语义规则，且每条规则的形式为公式 3 所示，则称属性 b 依赖于属性 $c_1, c_2, \dots, c_k$ 。

$$b := f(c_1, c_2, \dots, c_k) \quad (3)$$

式中 b --A 某个综合属性或者继承属性；

c_n --文法符号 a 的属性。

为了实现对抽象语法树属性的计算，属性文法提供了三种属性计算方式，分别为依赖图，树遍历和一遍扫描，如下图 3-12 展示了所构建依赖图的结构。

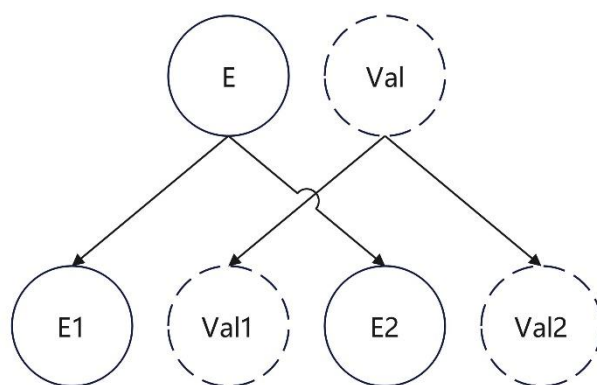


图 3-12 依赖图结构

Figure 3-12 depends on the graph structure

同时跟抽象语法树构建一样，由于 PLY 库没有提供与 Lex 与 Yacc 模块所对应的语义分析器，所以本文的交通互操作语义编译解析模型将采用手工编写的方法去实现语义分析模块，即在每一个 BNF 范式中添加相应的语义动作，对其所表示的文法符号进行处理。具体而言，首先确定整个语义分析模块的目标为实现符号表的建立和语法树静态语义检查，其中前者需要遍历语法分析所产生的抽象语法树，同时提取出每一个节点所具有的属性值，同时储存在某一特点数据结构中，而后者需要对符号表中所储存的信息进行处理，根据交通互操作语言语义规范检查是否存在语义错误，包括类型错误，变量合法性错误以及唯一性错误等。为此本文根据上述需求，为交通互操作语言语义分析模块设计了一个语义分析库，其中包含所有手工设计并用于实现语义分析的类，包含 SyntaxTree 类，SyntaxNode 类，SymbolTable 类，Symol 类等，具体结构如图 3-13 所示。

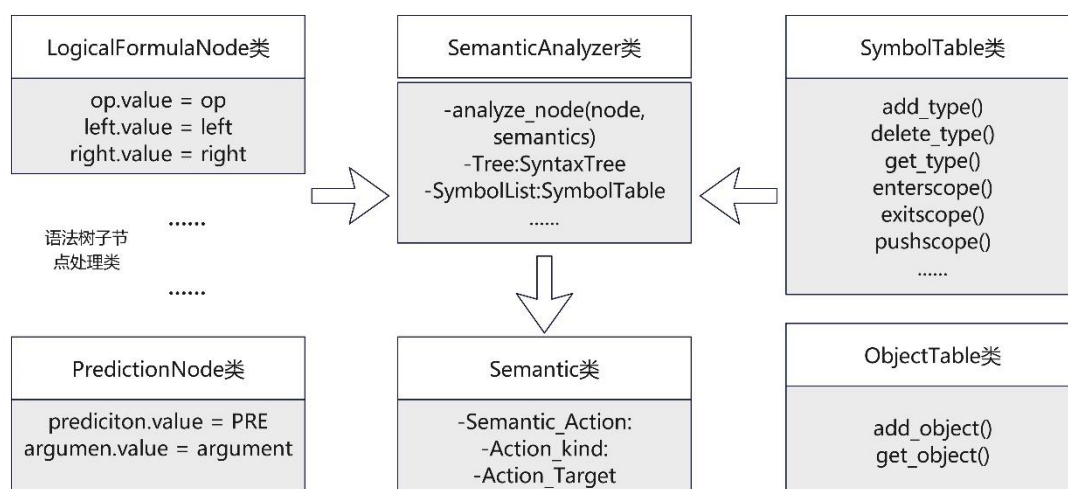


图 3-13 语义分析库组成类图

Figure 3-13 The class diagram of the semantic analysis library

其中 `SemanticAnalyzer` 类是最为核心的类，负责集成语义分析库的类然后对所处理的交通互操作语言进行符号表构建，以及相应的静态语义检查，并且返回 `Semantic` 实例作为其输出。`Semantic` 类负责收集 `SemanticAnalyzer` 所获得的语义信息，将其作为整个语义分析模块的输出，提供给语句执行模块调用相关的预设函数进行执行，不同的交通互操作语句所使用到的相关执行函数是不同的，在本文里简单的将其引导词不同划分为谓词语句，函数语句和动作语句，在实际情况中划分可能更为细致和负责，因为对函数语句和动作语句而且，其不同关键词的处理逻辑互不相同，因此为其设置了 `Action_kind` 和 `Action_Target` 两个变量，以储存检查到的关键词名称，以及动作语句可能涉及到的预期目标。其次是语法树子节点处理类，这其实就是进行对所生成抽象语法树按照不同的子节点类型进行静态语义检查，同时将不同节点的语义信息储存入语法树依赖图中传递给 `Semantic` 类，然后 `SymolTable` 类和 `ObjectTable` 类主要用于存储和管理编译过程中遇到的所有符号（如变量名、常量名）及其相关信息（如类型、作用域）。

3.4.2 语义分析模块测试

通过对交通互操作语言语义的解析，本文已经完成了对交通互操作语言语义解析器的构建，为验证所设计语义解析器的效用，本文将进一步对其解析交通互操作语言的效用进行分析，下图 3-14 展示交通互操作语言语义解析器的工作流程。

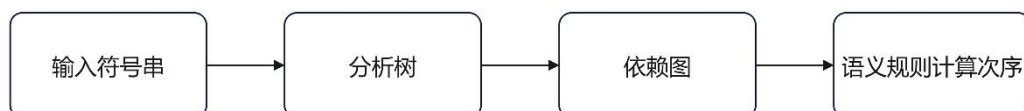


图 3-14 语义分析器工作流程

Figure 3-14 Semantic Analyzer Workflow

为了对设计好的语义解析器进行测试，本文将在程序中显式的展现出交通互操作语言的抽象语法树，并测试所手工设计类应用在解析语法树上的效果。语义解析器会遍历整个抽象语法树节点，获取每个节点所包含的语义信息，同时进行语义检查，检测是否有语义错误，然后返回所需要的语义信息，如下图 3-15 展示了对展示了对下某一具体交通互操作语句进行语义分析所返回的结果。

```

交通互操作语句:
 $\exists \text{Car}(\text{IsOverSpeed}(\text{speed}(\text{Car})) \wedge \text{IsLightOff}(\text{light}(\text{Car})))$ 
抽象语法树:
QuantifierFormulaNode:
Quantifier:  $\exists$ 
Constant: Car
LogicalFormulaNode:  $\wedge$ 
AtomFormulaNode: IsOverSpee
VariableNode:
Variable: speed
Attached Constant: Car
AtomFormulaNode: IsLightOf
VariableNode:
Variable: light
Attached Constant: Car
语句类型: 查询, 关系符号: IsOverSpeed, IsLightoff, 目标: speed(Car), light(Car)

```

图 3-15 交通互操作语句语义解析

Figure 3-15 Semantic analysis of traffic interoperability statements

3.5 策略执行模块

交通互操作语言是一种通信语言，他是用来承载跨域喝域内交通主体之间的交互语义的，并不需要靠它去推动交通主体执行，只需要然交通主体在接受到交通互操作语言后能理解其语义，然后执行相应动作即可，所以为了同本文第 4 章的语义推理模型和第五章的仿真测试结合起来，本文在策略执行模块为交通互操作语言专门构建了一个执行函数库，通过接受语义分析模块传递的 Semantic 类调用执行函数库里不同语义动作所对应的执行函数，从而实现对相关知识的理解和相关动作的执行，具体步骤如下所示：

(1) 为每个非逻辑符号，定义相应的执行函数，并设定其对涉及的交通参数的处理逻辑，从而构建一个全面的执行函数库，并且所设计的执行函数返回值应当匹配非逻辑符号的类型。

(2) 接受语义分析器所产生的 Semantic 类，根据语义信息的不同，通过选用不同的执行函数进行处理，同时执行函数会根据需求将语义信息输入到语义推理模型。

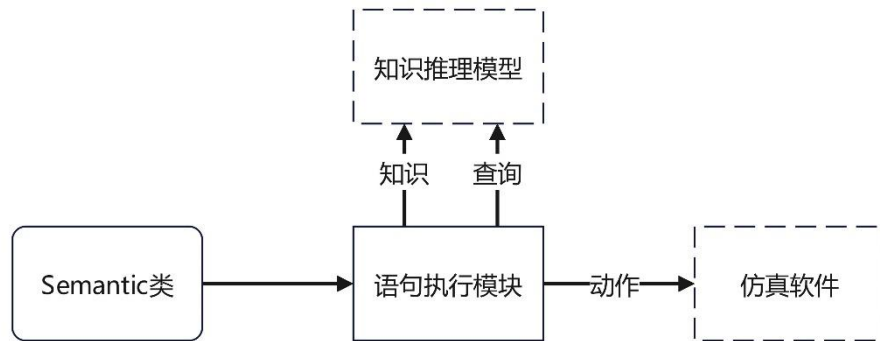


图 3-16 语句执行过程

Figure 3-16 Statement execution

本文设计的语句执行模块充当不同模块之间连接的作用，它将交通互操作语言编译理解模型与知识推理模型以及仿真软件桥接起来。这一模块的设计目的是为了确保不同模型之间的无缝连接和信息流的准确传递。在该模块中，本文构建了一个执行函数库，如图 3-17 所示，该库内部含有多个函数，每个函数都对应于相应的函数和动作关键词。这些函数的设计和实现，是基于本文前面章节对交通互操作语言语法规则和词法规则的深入分析，每一个函数和动作都是为了执行一项特定的任务，比如路况信息的更新、交通流量的计算、信号控制逻辑的实施等等，根据返回 Semantic 实例中储存的语句类型，选择对应函数执行相关语义。这样的设计允许模块对交通系统中发生的事件做出快速响应，并进行相应的处理，为了验证语句执行模块的有效性和可靠性，本文将不再单独对其进行测试，而是采取综合测试的方法，即在第五章的仿真验证中，一同测试了它与知识推理模型和仿真软件之间的集成，确保在模拟的交通环境中，所有的模型都能够正常工作，并且按照设计的预期交互。

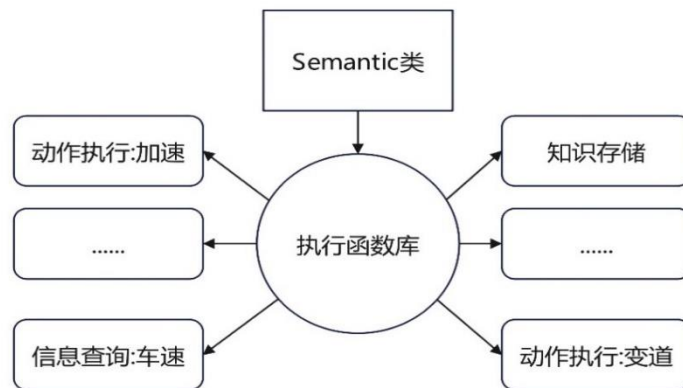


图 3-17 语句执行过程

Figure 3-17 Statement execution

3.6 本章小节

本章完成了对交通互操作语言语义解析模型整体框架的构建，然后基于这个框架，首先对比选择了合适的解析工具，然后按照模块化的结构，依此设计实现了以交通互操作语言词法规则为基础的词法解析模块，以交通互操作语法规则为基础的语法解析模型，以交通互操作语义规则为基础的语义解析模块，以及连接语义推理模型的策略执行模块，通过语义解析模型，实现了对交通互操作语言语义信息的抽取。

4 交通互操作语言语义推理模型

上一章详细探讨如何构建交通互操作语言的语义解析模型，这为本章的内容奠定了基础。通过语义解析模型，已经能够将交通互操作语句转换成交通主体能理解的内部表示，使得交通主体理解了交通互操作语句里所包含的语义，接下来进一步需要基于获得的语义，研究是否依此进行语义推理。本章的目标是设计和实现一个语义推理模型，该模型能够对已解析的交通互操作语句进行深入分析，并据此作出决策，这一过程不仅依赖于语句本身所包含的信息，还依赖所设计的交通领域的知识库和预定义的逻辑规则。

4.1 交通互操作语言语义推理模型设计

本章所提出的语义推理模型是在第三章语义理解模型上构建的，其希望通过交通互操作语句获取到的知识，对整个交通场景做进一步的智能理解和推理。其模型框架如图 4-1 所示。首先，模型的基础需要是构建一个自主式交通系统规则库。这个规则库要涉及各种可能发生的交通状况，在此基础上模型从语义理解模型返回的 **Semantic** 类中提取关键语义信息，调用对应规则库，送入到相应规则库中的 **Rete** 网络进行精确的模式匹配。**Rete** 网络是一种高效的模式匹配算法，能够快速地对大量规则进行匹配，找出适用的规则^[41]。一旦找到匹配的规则，系统便激活这些规则，促使交通主体执行相应的动作。以此语义推理模型能够实现对交通互操作语言语义的深层次理解和智能化推理，从而大大提高自主式交通系统的安全性、效率和灵活性。

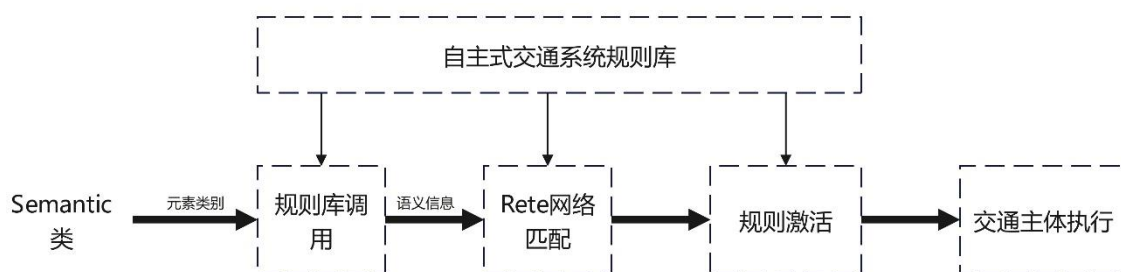


图 4-1 逻辑推理模型框架

Figure 4-1 Logical Inference Model Framework

4.2 自主式交通系统推理规则研究

4.2.1 交通知识划分

本体论在哲学中探讨实体及其相互关系的本质^[42]，而在人工智能和计算机科学领域，本体学已被用来管理和表达数据间的复杂关系。随技术进步，本体学的应用超越了传统专家系统，展现出在知识表示方面的丰富表达力。通过定义类、属性、函数、公理及实例，本体学深入描绘知识库中对象间的细致关系，特别适用于规则众多、关系复杂的场景，有效组织信息。知识表述依赖于类和属性，其中类聚合共性对象，明确其名称、层级关系；属性分为关系属性和数据属性，前者描绘实体间相互关系，后者定义实体特征或值。这种结构化知识表示不仅优化了信息组织，也提升了检索与推理效率。

在自主式交通系统中，推理规则本质上是从整个交通系统的一些知识推导出其他知识的逻辑过程，为了清晰地界定这些推理规则所依赖的交通知识，本部分将借鉴本体论的方法学，根据推理需要对自主式交通系统里道路交通的知识进行分类和划分，以此不仅有助于深入理解系统中各属性之间的逻辑关系，而且也为构建更为精确和高效的推理机制提供了坚实的基础。通过这种细致的知识划分，我们能够更好地捕捉到自主式交通系统的复杂性和多样性，从而为规则的设计和优化提供更加科学的指导，如下图 4-2 所示为对道路交通知识进行的类型划分。

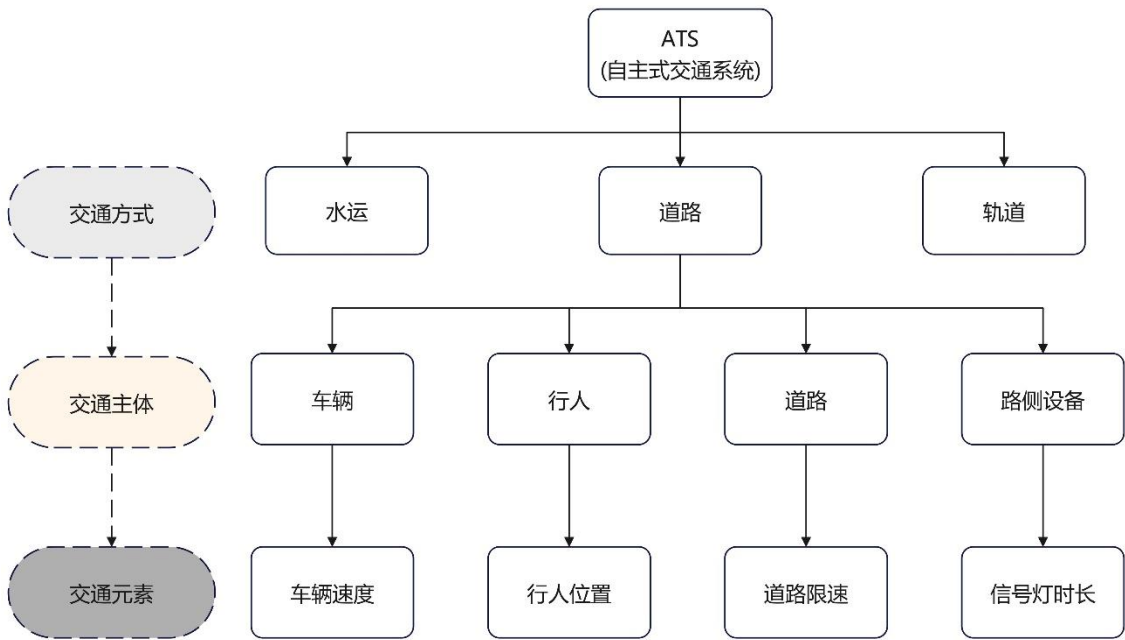


图 4-2 道路交通知识划分

Figure 4-2 Classification of road traffic knowledge

而进一步需要明白道路交通知识所可能包含属性，将所有类型划分为逻辑属性、数值属性和实体属性三种，这种分类方法极大地促进了对交通系统复杂性的理解和管理。逻辑属性关注于表达特定情况或条件的布尔逻辑，即某个特定的属性或状态是否成立，这对于描述交通规则启用条件或特定交通标志的存在与否至关重要。数值属性为描述交通系统的量化指标，包含了速度、距离、时间等关键参数，这些属性对于描述行程时长、判断碰撞风险等有着不可或缺的作用。实体属性则直接指向道路交通场景中的具体对象，如车辆、行人、信号灯等，这些属性帮助系统识别和区分交通主体及其动态，如表 4-1 所示，展示了部分道路交通知识的属性类别。

表 4-1 道路交通知识属性类别

Table 4-1 Categories of road traffic knowledge attributes

属性类别	元素名称	交通主体	属性取值
实体属性	CarLine	Vehicle	Line1/Line2/Line3
	Light	Trafficlight	Red/Green/Yellow
	Location	President	Dimension
	Speed	Vehicle	Double
数据属性	Duringtime	Trafficlight	Int
	Number	Road	Int
逻辑属性	OnLine	Vehicle	True/False

4.2.2 推理规则设计

本文所设计的自主式交通系统规则将会通过本文第二章所介绍的交通互操作语言进行表示，其是一种以一阶逻辑为基础的语言，在处理逻辑问题时有着相对的优势。

本文所设计的推理规则应遵循的具体结构如表 4-2 所示，其中 **Rule** 表示规则的名字，为每一条交通规则所独有的，具有唯一性，通过这个名字对相应交通规则进行调用和修改，如上表右侧展示的就是名为道路情况的交通规则，而 **Attribute** 则表示该交通规则所涉及到的交通元素，可以为空也可以有多个值，如右侧所示，**RondCondition** 所涉及的道路元素为 **Road** 交通主体下的 **Friciton** 属性，会根据结果对知识库里的元素进行修改，而 **LHS** 和 **RHS** 则分别表示规则所对应

的前提和推论，若满足 LHS 则推出 RHS，如若满足 Iswet(condition(Lane_1))(道路 Lane_1 潮湿为真)，则推出道路摩擦元素为易滑，将 slipper 赋值给 friction(Lane_1) 最后通过 End 标识整个规则的结束，至此完成一条规则的设计。

表 4-2 推理规则结构示意图

Table 4-2 Schematic diagram of the structure of inference rules

规则结构	规则示例
Rule Name	Rule RoadCondition
Attribute	Lane.friction
When	When
LHS	Iswet(condition(Lane_1))
Then	Then
RHS	slipper→friction(Lane_1)
End	End

此外对于整个自主式交通系统而言，将所有规则简单汇总到一个单一的规则库中是不现实且不合适的，因为不同的交通主体在不同的交通场景下所应该匹配的规则是不一样的，诚然整合所有规则会使得匹配规则的完备性得到保障，但是这样会大大增加了所构建推理网络的复杂度，增加的响应时间，交通主体执行某一个简单的交互需求，就需要遍历整个庞大的交通规则库，因此自主式交通系统规则库应当被划分为针对整个交通系统各个方面的多个子集，这样可以使规则更加精确且具体化，同时还能根据技术和社会需求的变化进行灵活调整。

这种分散化的规则管理策略简化了匹配过程，因为自主交通工具只需关注与之直接相关的子规则集，这样不仅加快了决策速度，还降低了计算复杂度，提升了系统的响应效率。此外，规则的更新和维护也变得更加简便，当某一特定区域的规则需要修改或更新时，只需针对相应的子规则集进行调整，而无需重新遍历整个规则库，从而使规则快速实时的根据实际情况进行变化，通过建立这样一个包含多个子规则集的规则体系，整个自主式交通系统的效率 and 安全性得以提高，同时还促进了不同交通参与者之间的有效协调和互动。

因此本文对自主式交通系统的规则库构建将会基于层次状态机理论^[43]将其划分为两层。顶层状态机将会按照不同的交通方式进行划分，分别划分为道路，轨道，水运三层，其次在这些顶层状态机之下，会按照不同交通方式内的交通主体划分多个子底层状态机，图 4-3 展示了整个规则库的架构，以及道路规则库的底层状态机。交通主体对互操作语句进行解析完之后，会根据语义分析返回的内容去调用相应的状态机去进行匹配操作，这样子能大大的缩短推理时间，保障自主式交通系统的实时性。

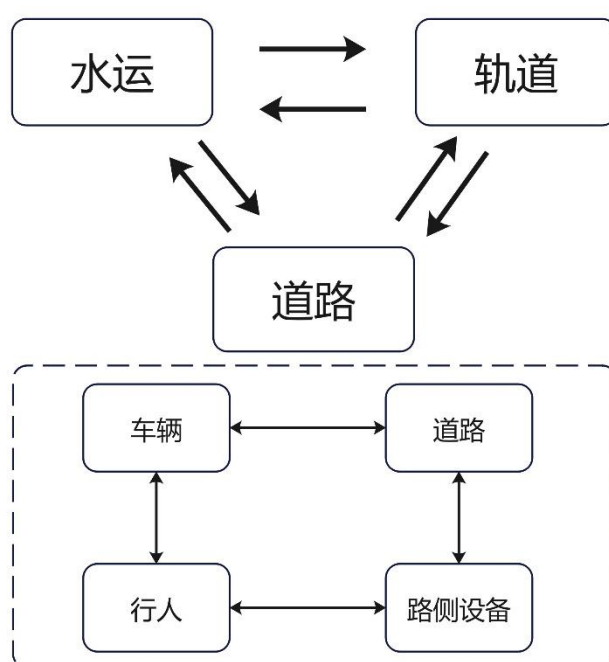


图 4-3 自主式交通系统规则库框架

Figure 4-3 Framework of the rule base for autonomous transportation systems

4.3 交通互操作语言逻辑推理算法研究

4.3.1 RETE 算法

Rete 算法是一种实现产生式规则系统的高效模式匹配算法，其常用于专家系统中规则引擎的实现，这种算法由 Charles L. Forgy 在 1974 年提出^[44]，该算法通过构建一个树状的 Rete 判别网络来进行模式匹配。其核心思想是根据内容动态构建分离的匹配项到匹配树中，以此大幅减少计算量，提高系统模式匹配的效率，而相对于其他算法其核心优势就在于其能够快速处理大量规则和事实之间的匹配问题，特别是在规则或输入频繁变化的情况下。Rete 算法可以分为两个主要部分：规则编译和运行时执行，规则编译是指根据预先设定的规则去生成 Rete 推理网络的过程，推理网络的结构如下图 4-4 所示，而运行时执行则是指实际运行中将事实送入 Rete 推理网络中进行选择的过程。

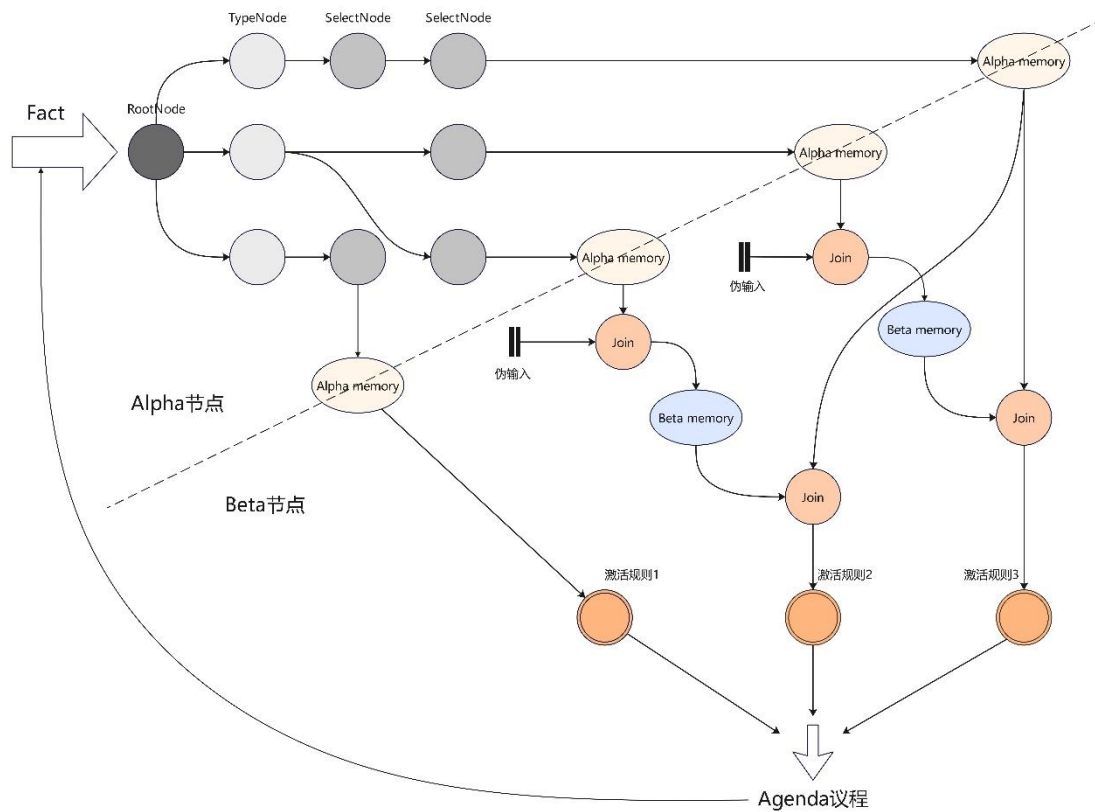


图 4-4 Rete 算法网络结构

Figure 4-4 Network structure of the Rete algorithm

Rete 算法的推理网络所包含的节点类型有 RootNode 节点(根节点), JoinNode(联合节点), TypeNode(类型节点), SelectNode(筛选节点), Alphamemory(Alpha 池), Betamemory(Bete 池)。整个推理网络根据不同部分实现功能的不同可以将整体分为 Alpha 节点和 Beta 节点。在 Alpha 节点,事实从根节点进入,通过类型节点对事实的对象类型进行筛选,确保只有符合其类型的事实能够进入相应的处理流程,提高处理效率,然后进入选择节点,根据规则模式中定义的属性条件对事实进行进一步的筛选,找出符合规则的每一个模式的集合,在 Alpha 池节点进行储存。

在 Beta 节点,其主要功能是综合和关联通过 Alpha 节点筛选后的事实。它负责处理不同 Alpha 节点输出的事实之间的逻辑关系和条件匹配,以实现复杂规则中的模式联合。Beta 节点接收来自一个或多个 Alpha 节点的经过初步筛选的事实,然后根据规则中定义的条件,检查这些事实之间的关系是否符合特定的模式。这包括比较不同事实的属性,如相等性、顺序或其他逻辑关系。满足条件的事实组合被进一步整合和储存在 Beta 内存中,为最终的规则触发准备好匹配的事实集合。通过这种方式,Beta 节点使得 RETE 算法能够高效地处理包含多个条件和事实的复杂规则,提高了整个规则引擎的匹配效率和性能。

总的而言, RETE 算法是一种启发式算法^[45], 其通过利用 AlphaMemory 和 BetaMemory 节点来存储已筛选的事实, 极大提升了规则匹配的效率。这种方法会在 Alpha 和 Beta 节点中保留之前的匹配状态, 允许算法在事实集发生小范围变更时避免重新进行大量计算。同时在 Beta 网络中, 由于多条规则存在可能共享相同的模式, 使得这些共享的 Beta 节点和 BetaMemory 不会执行相同重复判断, 实现了空间换时间的优化策略。例如, 如果一个 Beta 节点被 N 条规则共享, 那么该节点的处理效率就会提高 N 倍。并且 RETE 算法的匹配速度不会直接依赖于规则的数量, 因为输入的事实数据只在符合当前节点条件时才会继续在网络中向下传递, 避免了无关匹配的时间消耗。这些设计使 RETE 算法在处理规则引擎匹配时显得尤为高效, 即便是在规则数量众多的情况下也能保持较快的匹配速度。

4.3.2 RETE 算法改进策略设计

尽管 RETE 算法比起其他规则匹配算法具有其独特的优势, 但是本质上其还是一种牺牲空间复杂度, 换取时间复杂度的算法^[46], 使得在面对大规模的数据时推理速度和效率仍有进一步提升的空间。因此, 本文将结合自主式交通系统的推理需求, 对传统的 RETE 算法进行以下方面的改进和优化, 希望其能在处理大规模的交通主体交互需求时能发挥出良好的效果。

(1) 优化预设规则条件排列次序

Rete 网络在对规则进行编译时, 会依序为规则中的条件生成 Alpha 节点, 这就会导致其整体网络的构建是与其预设规则中条件的次序是有关联的。每一个 Beta 节点都直接或者间接联通多个 Alpha 节点, 若对应规则 1 和规则 2 而言, 其条件集合里存在相同的条件, 那么按理而言在 Rete 网络构建时, 可以通过共享其相同条件的 Alpha 或 Beta 节点来实现整体网络的优化, 但是在实际网络构建中, Rete 网络并不会去识别其预设规则之间是否存在可以共享的节点, 而是只会按照每条规则其预先设置时候的条件顺序对节点进行构建, 若相同条件的次序不匹配, 那么 Rete 网络就会对同一个条件生成多个不必要的 Beta 节点, 造成资源的浪费。

结合上述问题, 本文尝试是否能在 Rete 网络的构建阶段将规则的条件次序纳入考虑范畴, 通过为预设规则的条件集合设置合理的顺序从而提高整个 Rete 网络里阶段的共享程度, 从而达到降低占用和提高效率的目的, 因此本文提出一种条件排序算法, 通过对不同预设规则的条件进行合理排序使得不同规则共享同一节点, 从而提升 Rete 网络的效率, 优化前后 Rete 网络结构如图 4-5 所示, 其中展示了不同顺序下预设两个规则的 Rete 网络构建, 分别是两个规则所对应的条件集合分别是 $C2 \wedge C3 \wedge C4$ 和 $C1 \wedge C2 \wedge C3$, 由图可以看出按照预设的条件顺序所构

建的 Rete 网络有着 2 个 Alpha 节点, 4 个 Beta 节点和 4 个 Join 节点, 而按照优化顺序进行构建的 Rete 网络则仅由 1 个 Alpha 节点, 2 个 Beta 节点和 3 个 Join 节点, 可见对预设规则条件集合进行顺序优化, 能有效的降低整个网络的冗余节点, 提升匹配速度。

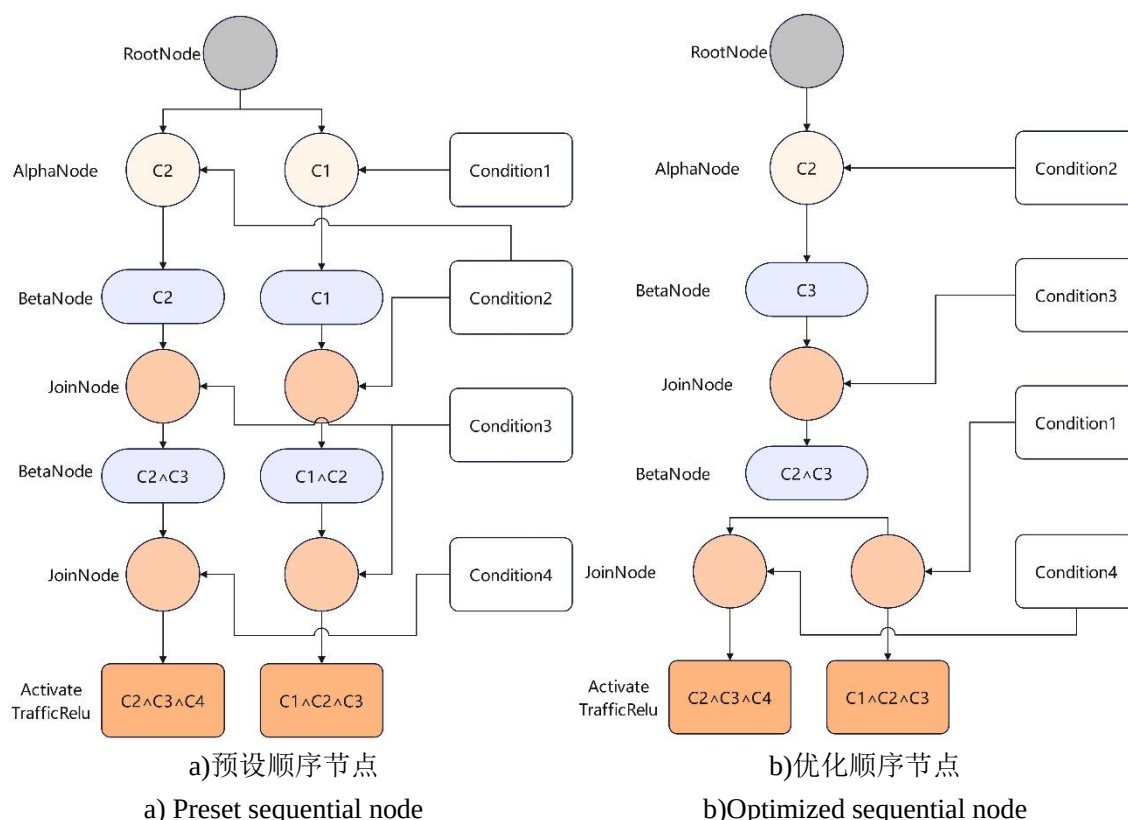


图 4-5 条件排列次序优化对比图

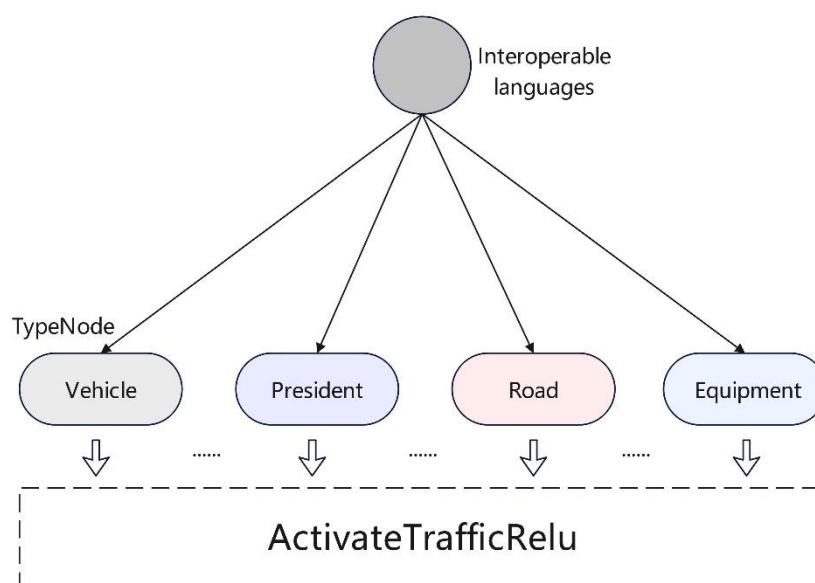
Figure 4-5 Comparison chart of condition ranking order optimization

(2) 优化 Rete 算法网络结构

对于传统的 Rete 算法网络结构而言, 事实通过根节点进入网络后会直接输入到 Typenoot 节点进行类型区分, 然后再依此通过 Alpha 节点和 Beta 节点, 最后激活相应规则。本文尝试对其整个网络结构进行优化, 通过增设缓存节点对整个流程进行改进, 缓存节点本质上可以看作是储存规则的内存, 其设立思想是依据“局部性原理”所提出的: 在较短的时间内, 程序倾向于重复访问相同的数据或资源, 即一旦某个数据被访问, 那么在不久的将来, 这个数据很可能会被再次访问。这在自主式交通系统里也是适用的, 在运行过程当中, 整个自主式交通系统感知到的数据往往都是趋于平稳的, 同样经常需要进行检测匹配的规则往往是加速匹配, 减速匹配等经常性进行的操作, 这就导致前一刻匹配到规则其很大概率会在之后的匹配频繁调用。

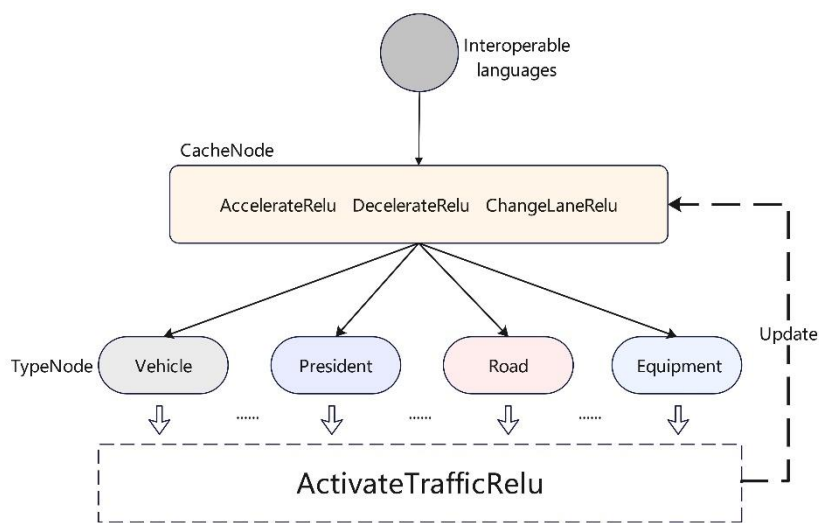
因此本文结合这个思想,提出“缓存节点”这一概念,这是一种介于根节点和 Alpha 节点之间的特殊节点,通过建立缓存机制来提升匹配效率。缓存节点的核心思想是预存储最近 N 条事实所匹配的规则索引值,当新的事实进入 Rete 网络时,它不会立即进入常规的类型筛选流程,而是首先被送往缓存节点。在这里,新事实会根据缓存的索引值顺序尝试与规则进行匹配。如果匹配成功,则该规则立即被激活,而且该规则的索引值会被移动到缓存列表的最前端,表示其高频使用的特性。如果在缓存节点中匹配失败,事实则会退出缓存节点,继续通过 Rete 网络的传统流程进行匹配,直到找到适用的规则。这样进行改进的优势在于其通过提高常用规则的匹配速度,从而提升整个系统的效率。由于缓存节点存储了最近匹配成功的规则索引,这意味着系统能够快速响应频繁出现的情况,减少了在整个网络中搜索匹配规则所需的时间和资源。此外,这种方法保持了 Rete 网络结构的完整性和匹配逻辑的一致性,不会因为引入缓存机制而牺牲准确性或可靠性。

总的来说,“缓存节点”的引入有效的提升了 Rete 算法的性能。通过智能地缓存和重用匹配信息,显著减少了匹配过程中的计算量,尤其是在自主式交通系统这种规则集和输入数据频繁变化的场景里,这种方法对性能的优化更为突出。并且缓存节点的设置并没有大幅改变整个 Rete 网络的网络结构,保持了 Rete 网络匹配的完整性,并有效的提升整个 Rete 网络的匹配效率,改进前后的 Rete 网络结构图如图 4-6 所示。



a)传统 Rete 匹配流程

a)Traditional Rete matching process



b)改进 Rete 算法匹配流程

b) Improve the matching process of the Rete algorithm

图 4-6 Rete 算法网络优化对比图

Figure 4-6 Comparison of network optimization of the Rete algorithm

4.3.3 RETE 算法改进策略实现

(1) 条件排序算法实现

为了实现对预设规则条件集合的排序，首先设置自主式交通系统规则库由规则集合构成{R1, R2, ..., Rn}，而每个规则则由条件集合构成{C1, C2, ..., Cn}，本文将会为规则库 RB 设置一个哈希表 M，其中不同的 Key 代表的是某一个具体的条件 C，而对应的 Value 这表示这个条件的出现次数 N，当输入好预设规则，完成对知识库的构建时，程序遍历知识库的每一条规则 R，将其拆分成其对应的 N 个条件 C，同时在遍历的过程中，实时对哈希表进行更新，包括增加新的 Key (新条件)，更新键值所对应的 Value(出现多次)。在遍历完整个知识库后，返回得到更新后的哈希表 M，然后据此对每个规则 R 的条件集合进行排序，按照每个条件所对应 Value 值的大小降序排列，然后再根据排序完的规则库 RB 进行规则编译，具体的实现步骤如下表 4-3 所示。

表 4-3 条件排序算法流程

Table 4-3 Conditional Ordering Algorithm Process

Algorithm 1 ConditionSorting Algorithm

Input: 未排序规则库 RB(Unsorted)

Output: 排序规则库 RB (Sorted)

```

1. function ConditionSorting Algorithm(RuleBase(Unsorted))
2.   M(Key: C=None, Value: N=0)
3.   for R in RB:
4.     for C in R:
5.       M(Key: C, Value: N+1)  $\leftarrow$  C OR M(Key: C, Value: 1)  $\leftarrow$  C
6.   For R in RB:
7.     {C1, C2, ..., Cn}  $\leftarrow$  Sort(Valuec1, Valuec2, ...Valuecn)
8.   return RB(Sorted)

```

(2) 结构优化算法实现

在构建缓存节点时候，采用双向链表实现 LRU 策略^[47]，为其构建一个哈希表 **M** 去表示缓存节点的内存空间，具体结构如图 4-7 所示，其用来记录缓存节点所储存的交通规则的储存位置信息的为 **Key** 值，通过对这个 **Key** 值的查询，能实现对规则的导向，从而达到快速访问的目的，同时应该整个缓存节点里的交通规则是根据其最近的激活时间进行排序的，为了表达这个次序关系，还设置了 **NewerPtr** 值和 **OldPtr** 值，前者指向下一个更新的哈希项，后者指向前一个更新的哈希项，在需要对缓存节点里交通规则索引进行更新的时候，只需要对着两者进行更新，既可以完成对整个缓存节点的更新，具体的实现步骤如下表 4-4 所示。

表 4-4 结构优化算法流程

Table 4-4 Structural Optimization Algorithms Process

Algorithm 2 Structural optimization Algorithm
Input: 事实集{F ₁ , F ₂ , ..., F _n }, 规则集{R ₁ , R ₂ , ..., R _n }
Output: 激活规则集{R _a , R _b , ..., R _m }
1. function Structural optimization Algorithm (Facts)
2. Enter the RootNode
3. For relu in CacheNode
4. Fact attempts to match rules
5. If Fact match relu
6. Activate matched Relu
7. Else
8. Enter the AlphaRoot
9. Enter the BetaRoot
10. Activate matched Relu AND Update CacheNode

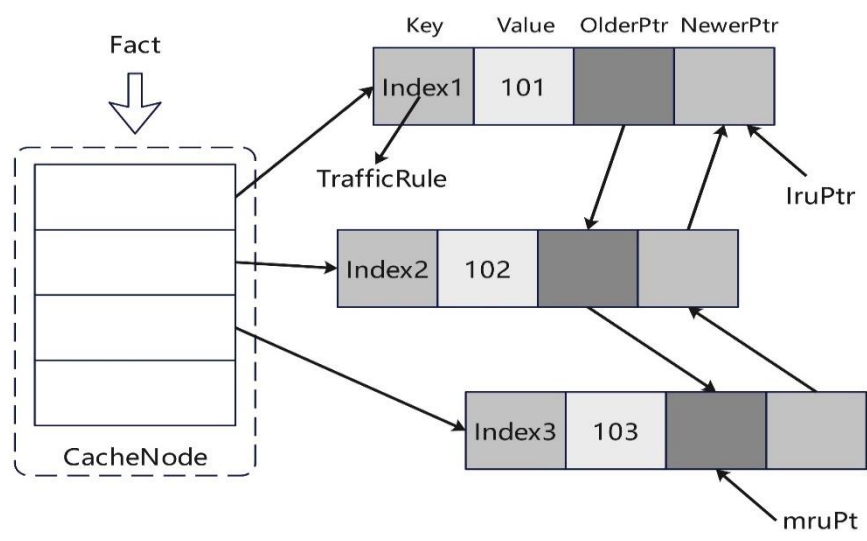


图 4-7 Cache 节点设计
Figure 4-7 Cache node design

4.3.4 改进 RETE 算法对比测试

为了对改进后的 Rete 算法性能进行测试，本节将会选择使用传统 Rete 算法与本文所设计的改进 Rete 算法进行对比实验，将会以两者处理不同数量交通规则所生成 Rete 网络占用的内存大小以及在接受交通互操作语言语义信息后的响应时间快慢作为实验评价指标。

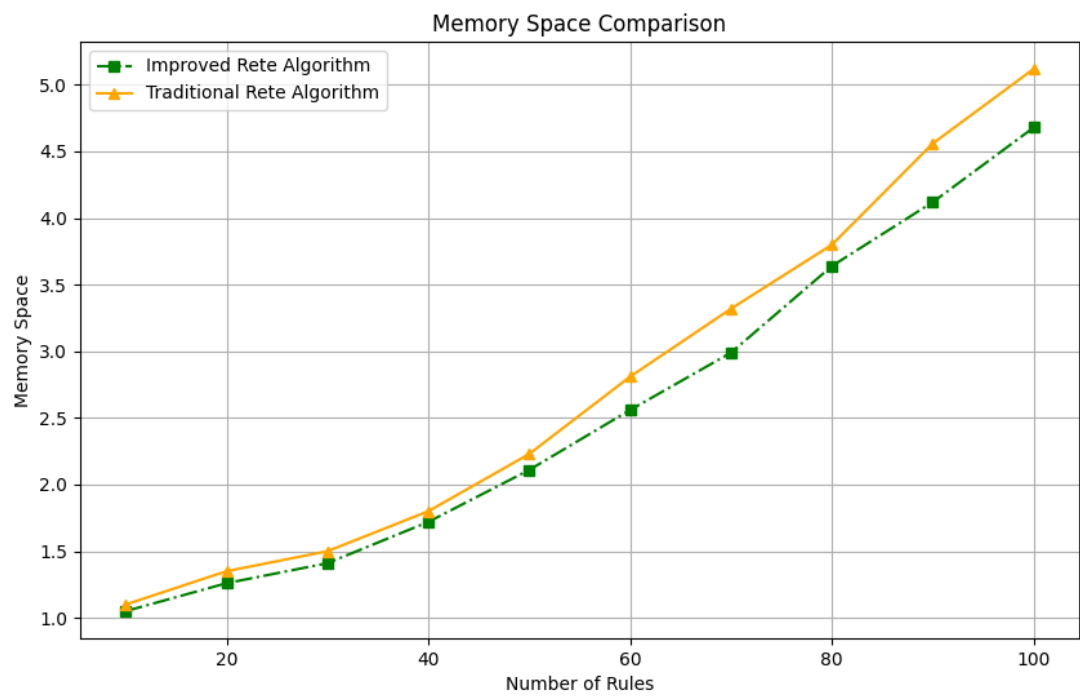


图 4-8 改进前面 Rete 算法内存占用
Figure 4-8 Improved memory usage of the previous Rete algorithm

改进前后 Rete 算法内存占用对比图如图 4-8 所示,通过比较可以看出,当输入的交通规则较少时候改进 Rete 算法在内存占用上与传统 Rete 算法拉不开差距,但是随着规则数量的不断增多,可以发现逐渐的改进 Rete 算法所占用的内存开始小于传统的 Rete 算法,当设置为 100 个规则时候,两者之差已经达到了 0.46MB,并且这个差距还呈现一种扩大的趋势,可见本文对 Rete 算法的优化能有效的降低整个 Rete 网络的内存大小,这对于自主式交通系统而言是很重要的,因为随着技术的发展,自主式交通系统需要处理的数据量正在急剧增加,这包括来自车辆、交通信号、环境感应器等多种源的信息,通过减少算法对内存的需求,使得这些系统就能够更快地处理更多的数据,提高其决策和反应速度,同时也能降低系统的硬件成本。

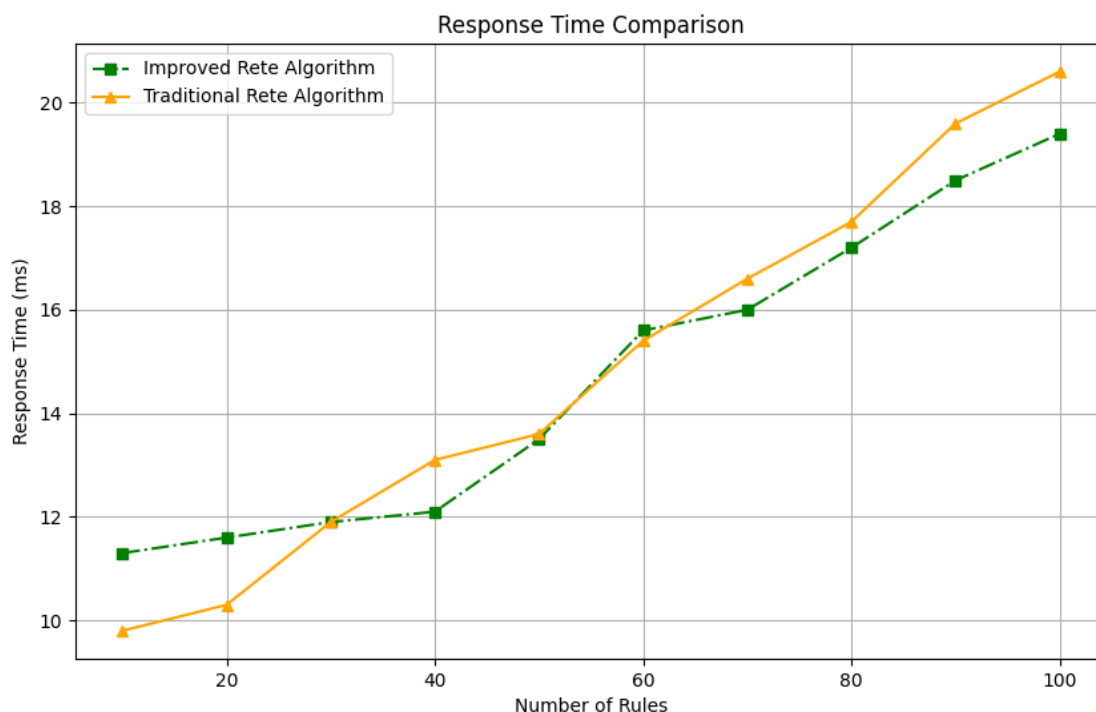


图 4-9 改进前后 Rete 算法响应时间

Figure 4-9 Response time of the Rete algorithm before and after improvement

改进前后 Rete 算法响应时间对比图如图 4-9 所示,通过比较可以看出,当输入的交通规则较少时候改进的 Rete 算法响应时长慢于传统 Rete 算法,但是随着规则数量的不断增多,可以发现逐渐的改进 Rete 算与传统的 Rete 算法的响应时间之差在不断缩小,并且逐渐最后改进 Rete 算法的响应时间以及快于传统 Rete 算法,并且当设置为 100 个规则时候,两者之差已经达到了 1.2ms,本文认为其可能原因可能是当规则个数较少的时候,整个 Rete 网络的匹配效率本身就快,这是增设其节点以及对其规则排序本身就是多余操作,导致整个网络匹配响应时间变慢,但当规则数量增多时,传统 Rete 算法的性能开始逐渐跟不上了,就使得其

与改进后的 Rete 算法响应时间差不断缩小，甚至慢于改进后的 Rete 算法。可见本文对 Rete 算法的优化能有效的降低整个 Rete 网络的匹配响应时间，这对于自主式交通系统而言同样是很重要的，因为它能够确保在面对高动态、多变的交通环境时，系统仍能保持高效和稳定的性能。通过减少匹配响应时间，自主式交通系统可以更快地做出决策和响应，从而提高交通流的效率，减少拥堵，降低事故率。

4.4 本章小节

本章完成了对交通互操作语言语义推理模型整体框架的构建，然后基于这个框架，首先通过交通知识的划分，为推理规则的设计明确了不同的交通主体和内容，在此基础上完成了对道路交通推理规则的设计，最后在此基础上，为了更好的匹配自主式交通系统的需求，对 RETE 推理算法进行结构和流程上的改进，根据对比实验结果表明，所设计的改进 RETE 算法相较于 RETE 算法而言在多规则的情况下，所占内存和匹配速度都较优。

5 仿真实验验证

本文的第三章和第四章分别针对交通互操作语言建立起了相应的语义解析模型和语义推理模型，但是单独对其中某一个进行测试是不够的，两者需要结合起来一起应用，因此需要进行相关实验，测试其联合使用的应用效果，但若在真实交通场景下进行实验则具有成本高，不安全等问题，因此本文选择通过设计仿真实验来测试验证本文所设计模型的有效性，主要测试在具体交通场景下交通主体对交通互操作语句语义的理解程度和相应语义动作的执行程度。

5.1 实验环境搭建

5.1.1 仿真平台选择

仿真平台的合理选择对于交通互操作语义理解模型和语义推理模型的展示效果而言是至关重要的，其不仅能够为整个模型提供一个可控和可观察的环境，还可以提供大量的数据和场景，帮助提升整个模型的泛化能力和适应性。因此本文调研了当前前沿仿真实验中所采用的主流仿真软件，并进行了归纳总结如下表 5.1 所示，并从中进行选择实现本文测试的仿真软件。

表 5-1 仿真软件调研
Table 5-1 Simulation software survey

仿真平台	功能	优势
CARLA	开源自动驾驶仿真平台	开源、高度可定制、丰富的环境和场景
AirSim	微软开发的开源仿真平台	高质量视觉仿真、多平台支持
Autoware Simulator	面向 Autoware 自动驾驶系统的仿真器	与 Autoware 紧密集成
LGSVL Simulator	基于 Unity 引擎的高性能仿真平台	支持多种传感器模拟并集成第三方软件

为了更好的实现对仿真其中 CARLA 仿真器以其开源性和高度可定制的特点，使其在车辆仿真领域中独居优势^[48]。不仅基于强大的虚幻 4 引擎，提供了高保真度的视觉渲染和物理模拟，CARLA 还支持丰富的环境设置和动态场景生成，允

许在多变的天气条件和复杂的城市布局中测试和优化自动驾驶算法。其服务器-客户端架构设计，确保了仿真平台的灵活性和扩展性，并且提供了丰富的 Python 和 C++的 API 接口，允许开发人员根据自己的需求自由控制车辆、传感器配置和交通流模型，以适应不同的测试需求。

相比其他仿真软件，CARLA 的另一大优势在于其实现了对其他自动驾驶模块的深度融合^[49]。通过提供与 ROS 和 Autoware 的无缝连接能力，CARLA 不仅能模拟车辆动力学和传感器输出，还能在仿真环境中直接应用和测试现实世界中的自动驾驶系统。此外，CARLA 的 Traffic Manager 模块能够模拟极其真实的交通环境，加上其提供的大量预设城市地图和传感器模型，大大加速了从理论到实践的转化过程。

综上所述，本文将选用 Carla 仿真软件来进行交通互操作语言语义理解模型和语义推理模型测试环境的搭建，如图 5-1 所示为 Carla 仿真软件的整体框架。

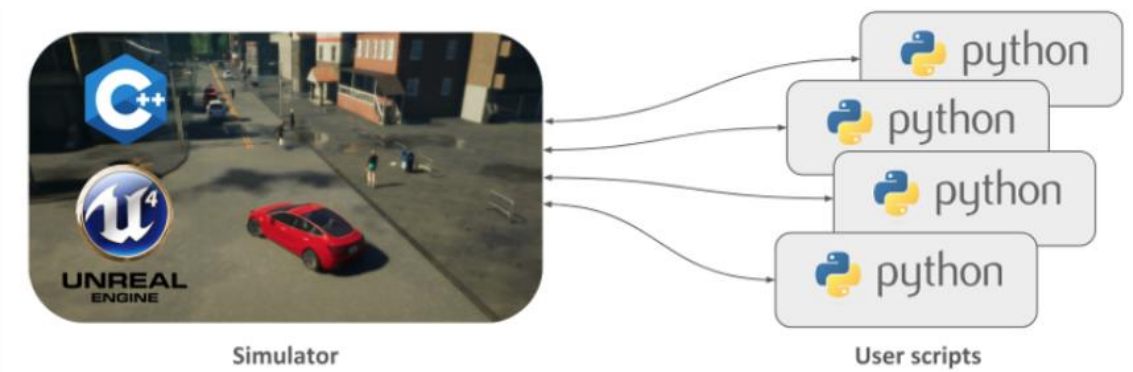


图 5-1 Carla 仿真软件框架

Figure 5-1 Carla simulation software framework

5.1.1 仿真环境的搭建

本文的仿真实验环境将会在 Window 系统下进行搭建，具体的搭建环境配置如表 5-2 所示。

表 5-2 仿真实验的环境配置

Table 5-2 Environment configuration of the simulation experiment

硬件/软件	平台
CPU	AMD Ryzen 9 7945HX with Radeon Graphics
GPU	NVIDIA GeForce RTX 4060
内存	32GB (16GB×2)

表 5-2（续表）

硬件/软件	平台
操作系统	Windows10
Carla 版本	Carla0. 9. 15
Python 版本	3.7.1

5.1.2 QT 界面开发设计

为了更好的对实验进行控制，以及对交通互操作语言解析推理过程的展现，本文将在 Carla 仿真软件的基础上，基于 PyQt5 的框架设计交通互操作语言语义理解平台界面。PyQt5 是一个用于创建图形用户界面的跨平台工具套件，它将 Qt 库的功能绑定到 Python 语言，Qt 是一种广泛使用的 C++ 框架，用于开发具有丰富图形界面的应用程序^[50]。如图 5-2 所示，所设计的 QT 界面主要包括四个主界面，分别是所选车辆视角界面，车辆状态信息界面，车辆实时知识库界面，以及命令执行过程界面，同时为了便于交通互操作语句输入为用户提供了互操作语言输入框，以及 4 个可选功能按钮。其中所选车辆视角界面和车辆状态信息界面同 Carla 仿真软件相联通，能实时获得 Carla 仿真里，所选中车辆的视角图像，并且将展示到解析平台中，同时获取所选中车辆的加速度，速度和方向盘转角的具体数值，绘制成图进行处理展示。而对于车辆实时知识库界面，这里展示的是储存的是车辆在实际运行过程中以交通互操作语言进行储存的交通知识，包含天气状况，最近交通信号灯状况和车辆目前所处车道等，会实时进行更新。而命令执行过程界面则是详细记录了每一条交通互操作语句从输入到输出的解析过程，将其展示在界面里给用户进行分析。而 4 个功能按钮的作用分别是输入知识，切换不同的车辆，切换不同的车辆视角，启动/暂停 Carla 仿真。综上，本文将基于这个互操作语言解析平台进行仿真验证。

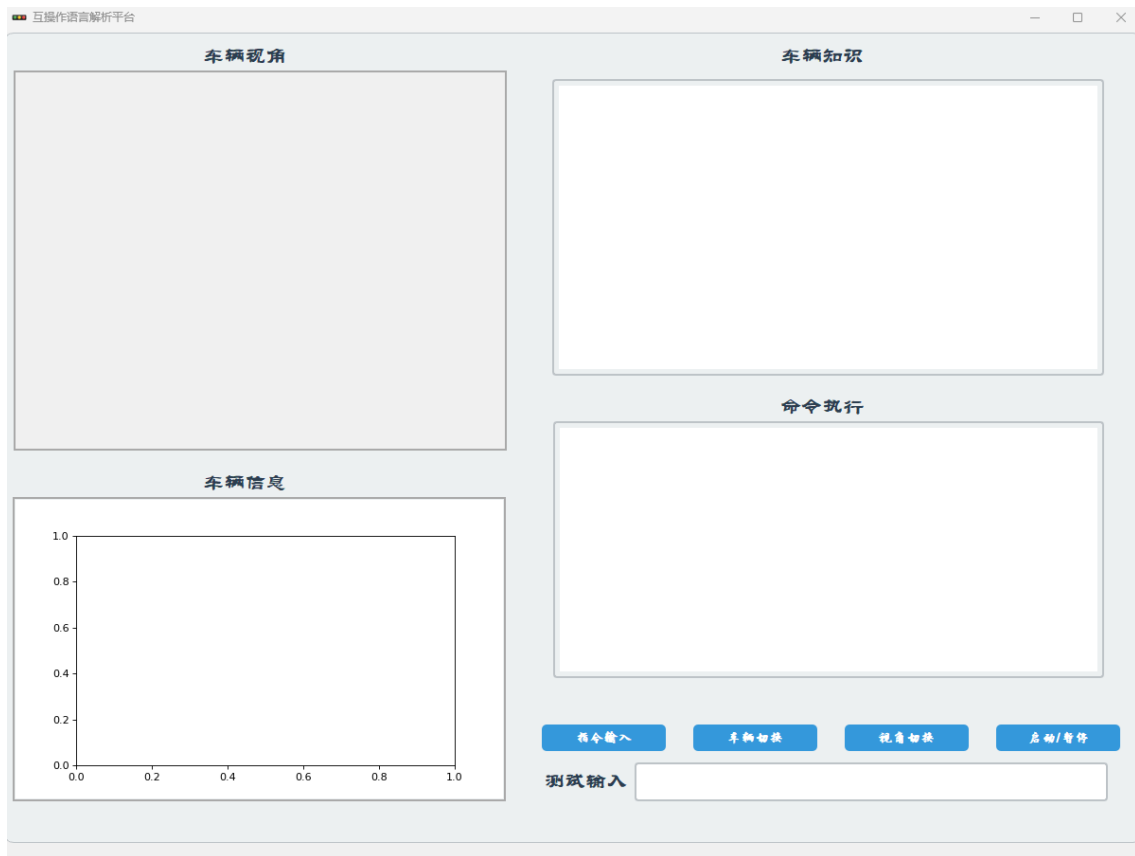


图 5-2 互操作语义解析平台界面

Figure 5-2 Interoperation semantic parsing platform interface

5.2 仿真实验设计

5.2.1 实验场景选择与设置

本文仿真测试将选取 Carla 地图库里的 Town 10 虚拟交通场景作为仿真场景。为满足本文仿真测试要求，将会设置一辆主视角车辆作为自车，并设定 Carla 服务器端为同步仿真模式，以确保从交通互操作语义理解平台给交通主体输入的交通互操作语言指令，能实时同步 Carla 服务器端，从而保障整体仿真运行逻辑的同步。如下图 5-3 所示为本文主要测试的几种交通场景环境。

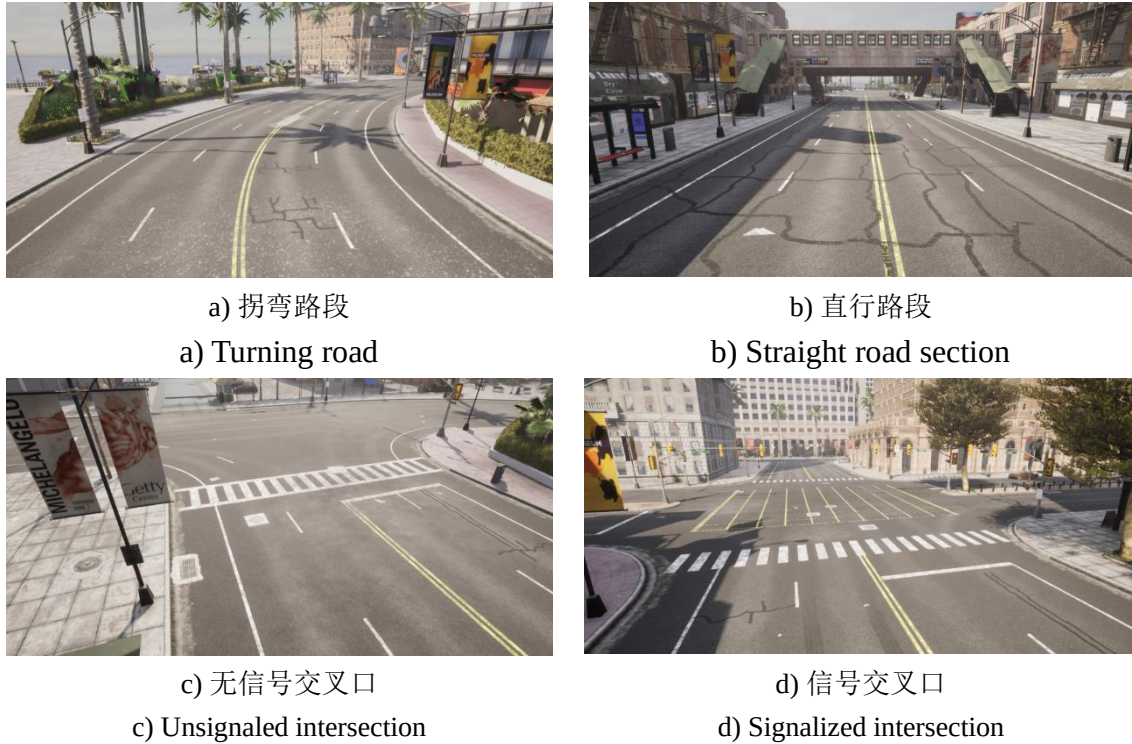


图 5-3 模型测试交通场景

Figure 5-3 The model tests the traffic scenario

5.2.2 测试方案设计

本文所设计的仿真实验目的是为了验证所设计交通互操作语义理解模型和语义推理模型是否能提取出交通互操作语言的语义，并且进行相应推理和动作执行，所以本章实验的关注重点是测试输入交通互操作语句对所控制车辆的影响，因为对于交通互操作语言来说，其基本组成单元为各类原子语句，因此本文将按照交通互操作语言原子语句类型划分来进行测试，通过输入不同类型的原子语句，如谓词，函数，动作类型的原子语句，在 Carla 仿真里观测控制车辆的状态变化来测试本文所设计模型的有效性。

(1) 谓词语句测试，设计测试案例来验证车辆是否能理解相应的谓词语句，同时需要实现对逻辑运算符的支持。

(2) 函数语句测试，设计测试案例来验证车辆车辆是否理解相应的函数语句，包括对车辆状态如速度，加速度的查询函数。

(3) 动作语句测试，设计测试案例来验证控制车辆是否能理解并执行指定的动作，并且同时会根据动作语句进行相应的推理。

5.3 仿真测试与分析

5.3.1 谓词语句

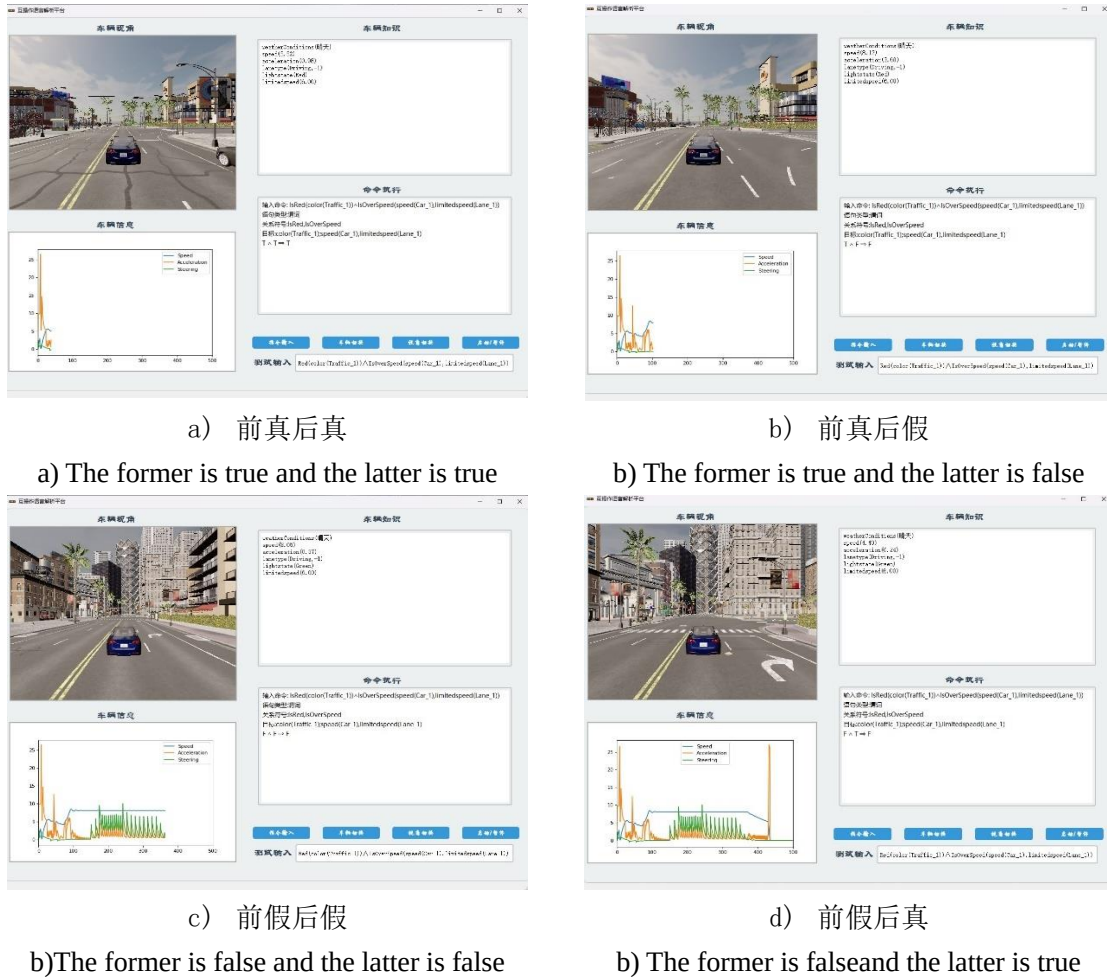


图 5-4 谓词语句测试

Figure 5-4 Predicate sentence test

如图 5-4 所示，通过交通互操作语言解析界面对交通互操作语言谓词语句进行测试，对于谓词语句而言，需要在确保其语法语义正确的基础上，检索相应交通知识，对语句的真假进行判断，并且不同于其他语句，对于谓词语句而言，其需要能支持通过逻辑连接词连接的进一步推理。如图所示，对所控制的车辆输入 $IsRed(color(Trafficlight_1) \wedge IsOverSpeed(speed(Car_1), limitedspeed(Lane_1)))$ 语句，其中 $IsRed$ 和 $IsOverSpeed$ 表示相应的谓词符号，返回的值会是 $True$ 和 $Flase$ ，同时在后续括号里个体变量表示了所进行对比的对象，在此包括对信号灯的颜色，车辆的速度，道路的限速，整个语句的含义就是判断控制离控制车辆最近的信号灯是否为红色并且车辆没有超速。对此测试其输入，在控制车辆收到所传递的交

通互操作语言后，其首先通过语义解析模型对语句进行解析，并且返回其语义信息，包括语句类型，关系符号，以及目标，控制车辆以此为依据对语句进行执行，调用相应谓词判断逻辑，检索车辆知识库，进行逻辑判断。所以可以看到，在进行语句输入之后，命令执行栏里相较于其他类型的语句会多出一行逻辑判断行，其表示所输入谓词逻辑的整体的真假关系。

5.3.2 函数语句

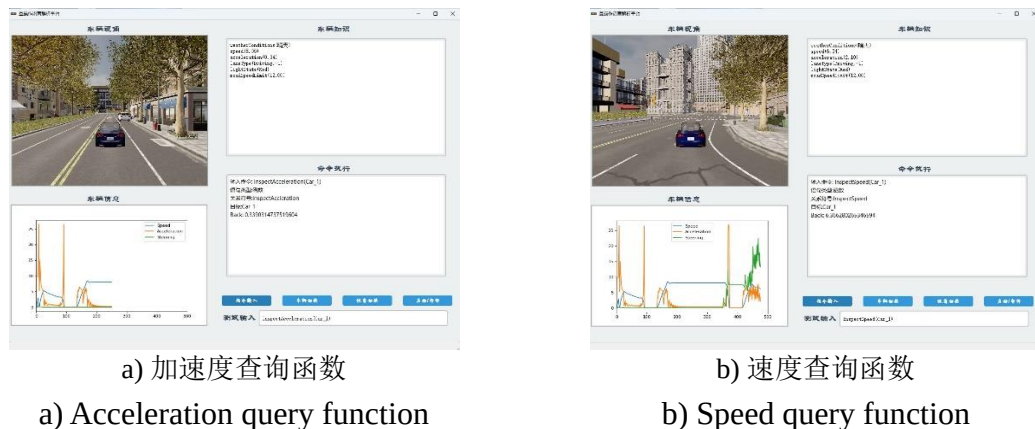


图 5-5 函数语句测试

Figure 5-5 Functional statement testing

如图 5-5 所示，通过交通互操作语言解析界面对交通互操作语言函数语句进行测试，对于函数语句而言，需要在确保其语法语义正确的基础上，主体能根据其语义内容，检索所涉及交通知识并把运算后得到的知识进行返回。以进行下一步推理。如图所示，对控制车辆输入 `InspectSpeed(Car_1)`和 `InspectAcceleration(Car_1)`语句，其中 `InspectSpeed` 和 `InspectAcceleration` 为预设的函数符号，分别返回对应主体的速度和加速度值，同时后续括号内个体常量 `Car_1` 确定所获取信息所对应的交通主体，整个语句的含义就是查询控制车辆的速度和加速度。对此测试其输入，在控制车辆收到所传递的交通互操作语言后，其首先通过语义解析模型对语句进行解析，并且返回其语义信息，包括语句类型，关系符号，以及目标，控制车辆以此为依据对语句进行执行，调用相应预设函数，检索车辆知识库，返回相应时刻的速度信息和加速度信息，所以可以看到，在进行语句输入之后，命令执行栏里相较于其他类型的语句会多出一行 `Back` 行，其后接所输入函数语句所返回的交通信息。

5.3.3 动作语句



图 5-6 动作语句测试

Figure 5-6 Action statement test

如图 5-6 所示, 通过交通互操作语言解析界面对交通互操作语言动作语句进行测试, 对于动作语句而言, 需要在确保其语句的语法和语义都正确的基础上, 及时根据其语义内容, 控制车辆完成相关操作的执行, 同时因为车辆的控制应当是实时的, 变化的, 所以动作语句在执行过程中仍然会进行多项逻辑推理, 所以本文分别测试了在不同场景下, 车辆执行加速动作语句的状况, 如图所示, 对所控制的车辆输入 `Accelerate(Car_1,15,0.5)` 语句, 其中 `Accelerate` 表示加速动作, 这在交通互操作语言中为预设的动作符号, 识别出来即可以判断语句是加速动作, 然后 `Car_1` 表示所执行动作的交通主体, 然后后续数字依次代表期望的目标速度和加速度, 即车辆要以多少的加速度加速到多少速度。测试其输入, 在控制车辆

收到所传递的交通互操作语言后，其首先对其语义进行理解，并且返回其语义信息，包含语语句类型，关系符号，以及目标，确定为动作语句后，会根据括号内预期参数去控制车辆去执行，但是可以发现，在不同的交通场景下对加速语句执行的情况是不一样的，在转弯，交叉口和到达限速的状况下，对车辆输入加速语句，车辆同时也会根据自身车辆知识进行判断，即是否能完成加速操作，如果不能则不会执行相应的操作，并且会返回不能完成的原因，即推理不满足的条件，当所有条件全满足，加速推理为 **True** 的时候，车辆才会进行动作的执行。并且可以看到，在进行语句输入之后，命令执行栏里相较于其他类型的语句会多出一行信息反馈行，其后接所输入动作语句的执行状况。

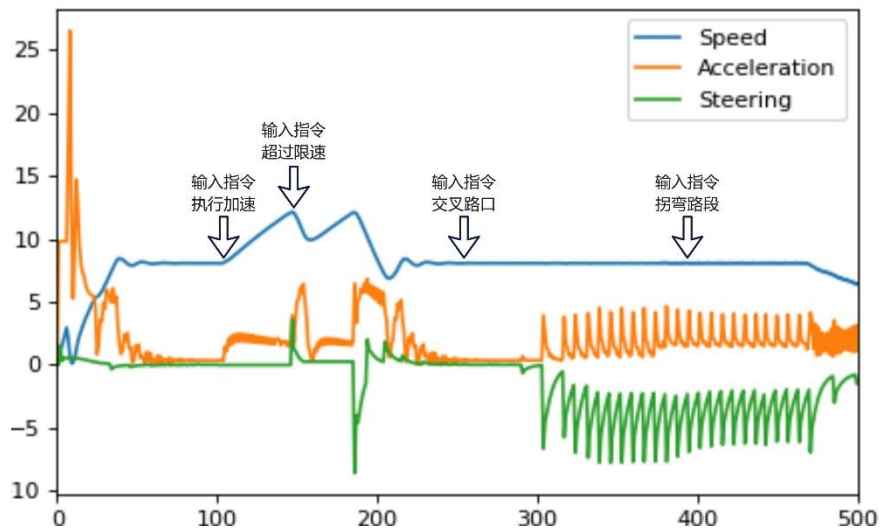


图 5-7 动作测试车辆速度变化图

Figure 5-7 Diagram of the speed change of the action test vehicle

为了深入分析并展示车辆在不同驾驶环境下对加速语句的响应差异，本文选取了一个测试周期中车辆状态的变化图，如图 5-7 所示，并在图中明确标注了交通互操作语言加速语言输入的具体时刻。在该测试周期内，共尝试输入了四次加速语句，但仅有第一次语句被有效地执行。从图中可以清晰地观察到，在第一次输入的加速语句时，由于控制车辆的其他推理规则都满足，车辆随即开始了匀加速行驶，这表明语句得到了正确的执行。然而，当车辆达到道路限速时，继续对其进行加速语句的输入不仅不会被执行，车辆速度反而会被自动调整以减速至限速以下，以确保安全合法。此外，当车辆在转弯时尝试加速，由于转向角度（Steer）不为零，推理模型会判断加速指令不满足条件，从而不执行加速指令。同样，在经过交叉口时，尽管发出了加速指令，但因为在交叉口进行加速不符合加速语句的逻辑推理，所以语句也不会被执行。

5.4 本章小结

本章首先搭建了模型仿真测试的实验环境，并对整体的搭建环境做了一个介绍，然后为交通互操作语言的不同类型的原子语句设计了仿真实验，测试所设计的交通互操作语言解析模型和推理模型是否能实现对交通互操作语言的语义理解，并且在 Carla 仿真里进行了可视化分析，同时为了方便用户进行控制，基于 PyQt5 框架设计了交通互操作语义理解界面。

6 结论与展望

6.1 研究结论

本文基于对交通互操作语言语法分析，提出了针对交通互操作语言的语义解析模型和语义推理模型，并在此基础上使用 Carla 仿真软件对其进行了仿真测试，验证了两个模型在仿真交通场景下的有效性，并得到了相关结论。论文的主要工作与结论如下：

(1) 开发了一个用于处理交通互操作语言的语义解析模型，这一模型依据第二章所介绍的交通互操作语言规范来设置相应的词法和语法。同时利用 PLY 库中的 Lex 工具和 Yacc 工具，设计并测试了交通互操作语言词法分析器和语法分析器。此外，手工设计实现了交通互操作语言的语义分析模块，实现了交通互操作语言的语义规则的检查，并通过设计策略执行模块的执行函数库，实现了解析后交通互操作语义信息的传递，进而联通了第四章的语义推理模型。

(2) 开发了一个用于处理交通互操作语义信息的语义推理模型，并将道路领域的交通知识按主体进行了分类。这一分类工作为后续的推理规则设计提供了清晰的指导，明确了各种交通知识的类别所属。在此基础上，为了更好地满足自主式交通系统的特定需求，对 RETE 推理算法的结构和流程进行了优化。通过对比实验结果显示，改进后的 RETE 算法在处理多条规则时，无论是内存占用还是匹配速度，均优于原始 RETE 算法。

(3) 基于 Carla 仿真软件和 PYQT5 框架开发了交通互操作语言语义理解仿真测试界面。同时针对交通互操作语言的不同语义动作语句设计了一系列仿真实验。通过仿真实验证明本文设计的交通互操作语言解析模型和推理模型能够对理解交通互操作语言的语义做出准确的理解和推理。

6.2 研究展望

虽然本文在交通互操作语言语义理解方法进行一定的研究工作，但是由于自主式交通系统整体比较复杂，其所涉及到的交通方式以及交通主体范围较广，类别较多，且本人的研究时间和科研能力有限，本文所仍有诸多方法存在不足，需要进行进一步研究和改进，主要包括如下几个方面。

(1) 本文只对道路交通领域的推理规则库进行了设计和测试，这对于实现自

自主式交通系统内部的互操作性与协同工作提供了初步的支持。然而，真正的自主式交通系统是一个包含铁路、道路、水运等多种交通方式在内的复杂交通系统。因此，随着未来研究的深入，需要为更多的交通方式设计推理规则库，通过建立一个综合性的、能够覆盖各种交通方式的推理规则库，为自主式交通系统奠定坚实的基础。

（2）本文仿真测试的交通场景所选择的是道路交通里比较典型的一些交通场景，在这些交通场景下进行交通互操作语义理解的测试确实能够验证整个模型有效性，但是对于自主式交通系统而言，其目标是让整个交通系统在任何场景都能实现自主化运行，此时，仅仅在经典的交通场景进行仿真测试就显得不足了，下一步需要在更为复杂，更为具体的交通环境中对模型进行测试与改进。

参考文献

- [1] 国务院关于印发“十四五”现代综合交通运输体系发展规划的通知[J]. 中华人民共和国国务院公报, 2022, (04): 8-28.
- [2] 本刊编辑部. 壮阔“十三五”[J]. 中国公路, 2021, (01): 13.
- [3] 李蒙恩. 自主式交通环境下的交叉口信号协调控制研究[D]. 长安大学, 2023.
- [4] 李子阳. 面向自主式交通系统架构的优化方法研究[D]. 哈尔滨工业大学, 2022.
- [5] Quaglietta E, Pellegrini P, Goverde R M P, et al. The ON-TIME real-time railway traffic management framework: A proof-of-concept using a scalable standardised data communication architecture[J]. *Transportation Research Part C: Emerging Technologies*, 2016, 63: 23-50.
- [6] Agbaje P, Anjum A, Mitra A, et al. Survey of interoperability challenges in the internet of vehicles[J]. *IEEE Transactions on Intelligent Transportation Systems*, 2022, 23(12): 22838-22861.
- [7] 刘诚臣. 考虑部件互操作的高速列车系统可靠性建模研究[D]. 北京交通大学, 2020.
- [8] Costanzi R, Fenucci D, Manzari V, et al. Interoperability among unmanned maritime vehicles: Review and first in-field experimentation[J]. *Frontiers in Robotics and AI*, 2020, 7: 91.
- [9] Kim D H, Choi K H. A framework for interoperability of vehicular telematics contents[C]//2014 IEEE International Technology Management Conference. IEEE, 2014: 1-4.
- [10] 刘光昀. 基于马尔可夫链的自主式交通系统架构可靠性评估[D]. 哈尔滨工业大学, 2022.
- [11] Huang X, Cen X, Cai M, et al. A framework to analyze function domains of autonomous transportation systems based on text analysis[J]. *Mathematics*, 2022, 11(1): 158.
- [12] 刘柯序. 基于知识图谱的编译器后端开发研究与实现[D]. 北京邮电大学, 2024.
- [13] Gough B J, Stallman R. An Introduction to GCC[M]. Network Theory Limited, 2004.
- [14] Lattner C, Adve V. A compilation framework for lifelong program analysis and transformation[C]//CGO. 2003, 4: 75.
- [15] 荆博祥, 王海荣, 王彤, 等. 多模态特征增强的双层融合知识推理方法[J/OL]. *计算机科学与探索*, 1-12[2024-04-13].
- [16] Gilpin L H, Ilievski F. Neuro-Symbolic Reasoning in the Traffic Domain[J].
- [17] Zhang J, Ilievski F, Kollaa A, et al. Utilizing Background Knowledge for Robust Reasoning over Traffic Situations[J]. *arxiv preprint arxiv:2212.07798*, 2022.
- [18] 张召霞. 面向无人驾驶车辆行为决策的知识库管理系统研究[D]. 中国科学技术大学, 2021.
- [19] Ward J R, Agamennoni G, Worrall S, et al. Extending time to collision for probabilistic reasoning in general traffic scenarios[J]. *Transportation Research Part C: Emerging Technologies*, 2015, 51: 66-82.
- [20] Souza C R C, Santos P E. Probabilistic logic reasoning about traffic scenes[C]//Towards Autonomous Robotic Systems: 12th Annual Conference, TAROS 2011, Sheffield, UK, August 31–September 2, 2011. *Proceedings 12*. Springer Berlin Heidelberg, 2011: 219-230.
- [21] Chalopin J, Changat M, Chepoi V, et al. First-order logic axiomatization of metric graph theory[J]. *Theoretical Computer Science*, 2024: 114460.

- [22] Li W, Sui Y. Variant quantifiers in L 3-valued first-order logic[J]. *Frontiers of Computer Science*, 2021, 15: 1-9.
- [23] 杨小龙.具有良好性质的密码函数的研究[D]. 中国科学技术大学, 2018.
- [24] 董林.时空关联规则挖掘研究[D].武汉大学,2014.
- [25] Chou G, Ozay N, Berenson D. Learning temporal logic formulas from suboptimal demonstrations: theory and experiments[J]. *Autonomous Robots*, 2022, 46(1): 149-174.
- [26] Li M, Wang L, Wu Q, et al. Diagnosis knowledge constrained network based on first-order logic for syndrome differentiation[J]. *Artificial Intelligence in Medicine*, 2024, 147: 102739.
- [27] 钟建.一种基于动态多元协同演绎的自动定理证明系统研究[D].西南交通大学,2021.
- [28] Atae S M. A frame and first-order logic solution for the wumpus world: Implemented in flora-2[J]. *Expert Systems with Applications*, 2023, 220: 119717.
- [29] Melo L T C, Ribeiro R G, Guimarães B C F, et al. Type inference for c: Applications to the static analysis of incomplete programs[J]. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 2020, 42(3): 1-71.
- [30] 朱丽佳.基于 NPU 的 P4 语言编译器的设计与实现[D].东南大学,2021.
- [31] Borsotti A, Breveglieri L, Raghizzi S C, et al. General parsing with regular expression matching[J]. *Journal of Computer Languages*, 2023, 74: 101176.
- [32] 李慧亮.基于 IEC61131-3 标准的 IL 语言编译器的设计与实现[D].西安电子科技大学,2014.
- [33] Postiglione A. Finite State Automata on Multi-Word Units for Efficient Text-Mining[J]. *Mathematics*, 2024, 12(4): 506.
- [34] Wang H, Zhao L, Li P. Nondeterministic finite automata based on quantum logic: language equivalence relation and robustness[J]. *International Journal of Approximate Reasoning*, 2021, 129: 20-40.
- [35] Leber Ž, Črepinšek M, Mernik M, et al. RNGSGLR: Generalization of the Context-Aware Scanning Architecture for All Character-Level Context-Free Languages[J]. *Mathematics*, 2022, 10(14): 2436.
- [36] 谢武平.云服务编程语言 Apla+ 及其实现方法研究[D].武汉大学,2017.
- [37] 蒋宏飞.基于同步树替换文法的统计机器翻译方法研究[D].哈尔滨工业大学,2010.
- [38] 宋健龙.面向数据库安全的 SQL 语句解析与翻译[D].北京理工大学,2015.
- [39] Habibi E, Mirian-Hosseiniabadi S H. Generating test as a web service (TaaWS) through a method-based attribute grammar[J]. *International Journal on Software Tools for Technology Transfer*, 2022, 24(4): 511-527.
- [40] 徐翊民.基于 SPICE 的 Verilog-A 编译器设计[D].华东师范大学,2023.
- [41] Geng D, Li H, Liu C. Tiny-UKSIE: An optimized lightweight semantic inference engine for reasoning uncertain knowledge[J]. *International Journal on Semantic Web and Information Systems (IJSWIS)*, 2022, 18(1): 1-23.
- [42] Katsumi M, Fox M. Ontologies for transportation research: A survey[J]. *Transportation Research Part C: Emerging Technologies*, 2018, 89: 53-82.
- [43] Wang X, Qi X, Wang P, et al. Decision making framework for autonomous vehicles driving behavior in complex scenarios via hierarchical state machine[J]. *Autonomous Intelligent Systems*, 2021, 1: 1-12.
- [44] Forgy C L. Rete: A Fast Algorithm for the Many Pattern[J]. *Many Object Pattern*, 1980.

- [45] 周爱民.基于规则推理的自主式交通系统架构诊断方法[D].哈尔滨工业大学,2022.
- [46] 宋双.基于 RETE 规则推理的告警关联分析子系统的设计与实现[D].北京邮电大学,2019.
- [47] 肖雄.工业应用中基于深度学习的缓存策略研究[D].南京邮电大学,2023.
- [48] Gutiérrez-Moreno R, Barea R, López-Guillén E, et al. Reinforcement learning-based autonomous driving at intersections in CARLA simulator[J]. Sensors, 2022, 22(21): 8373.
- [49] Li D, Okhrin O. Modified DDPG car-following model with a real-world human driving experience with CARLA simulator[J]. Transportation research part C: emerging technologies, 2023, 147: 103987.
- [50] 朱科.基于卷积神经网络的交通标志识别研究[D].西南交通大学,2019.